



# Deep Learning for Computer Vision

21 October  
2025

v. 2.1



# Seminar mixed with theory & practical sessions



## Theory Journey – How the course builds conceptually

### From Vision to Intelligence:

How machines learn to see

### Recognition & Tracking:

Teaching machines to follow what they see

### Deep Learning for Vision:

Neural networks that understand images

### Generative & Responsible Vision:

When machines start to create — and deceive

Applied & Creative Vision: **Designing with AI & looking ahead**

## The Practical Journey – Creating the Pixel Machine Lab

Image Manipulation with OpenCV · Mini Image Gallery

Face & Object Tracking · Dataset Annotation

Training Custom Models · Gesture Recognition · YOLOv9

**Micro Deep Dives:**  
Generative Vision · Diffusion · Ethics · Bias · AI Tools

Spatial AI and 3D Point Cloud Exploration with Open3D





Course Goals

# Object Detection

## YOLO



What is Deep Learning and why was it a breakthrough for Computer Vision

The architecture of Sight

Building a gesture recognition system that imitates a CNN

21 October  
2025







# Object Detection & YOLO

21 October  
2025







# Computer Vision Tasks

Classification



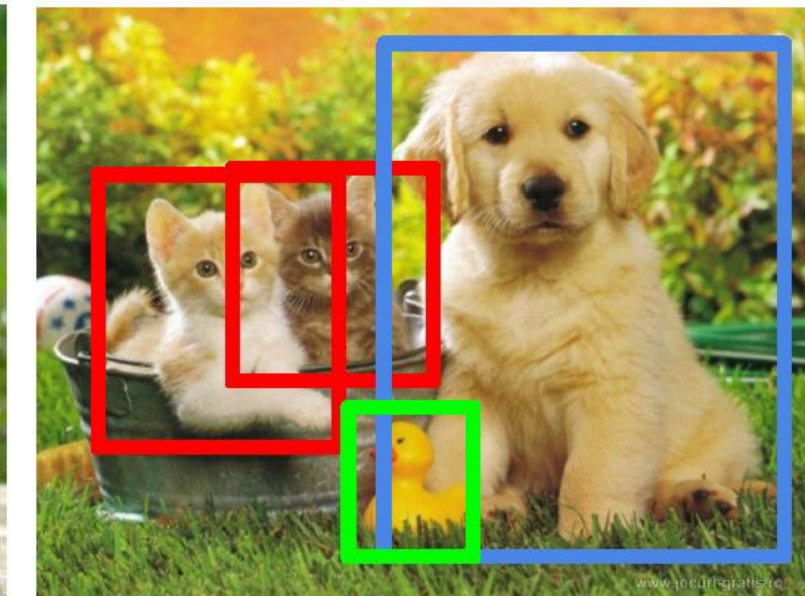
CAT

Classification  
+ Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

Single object

Multiple objects







# Computer Vision Tasks

Classification



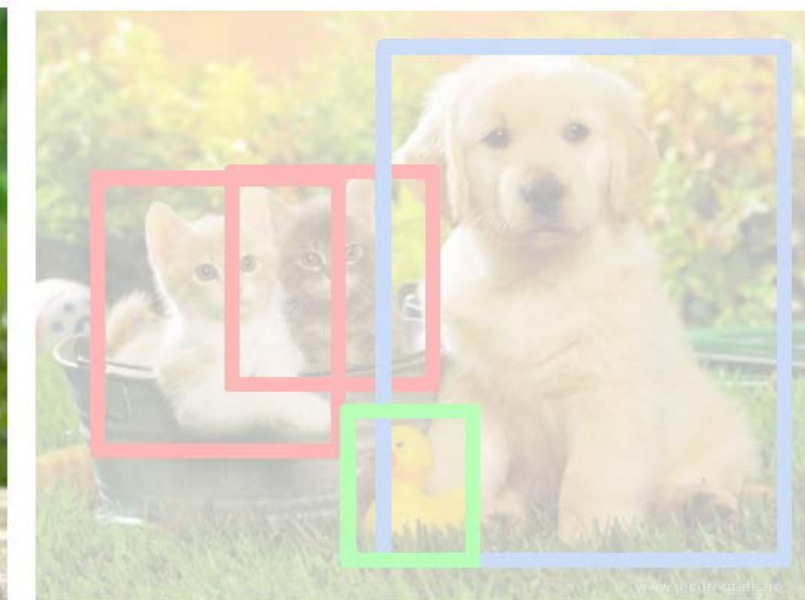
CAT

Classification  
+ Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

Single object

Multiple objects





# Classification & Localisation



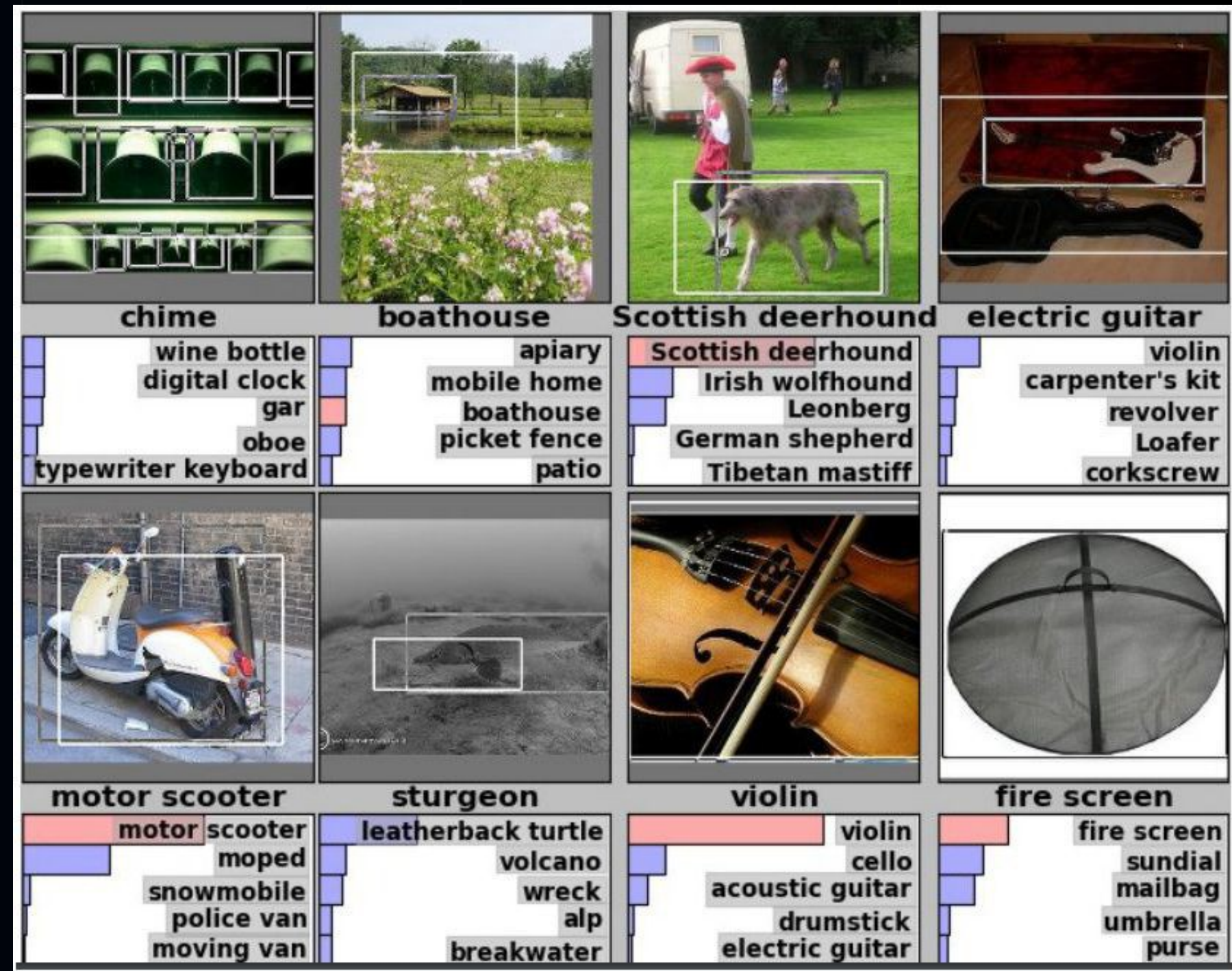
- Classification:
  - Input: Image
  - Output: Class label
  - Loss: Cross entropy (Softmaxlog)
  - Evaluation metric: Accuracy
- Localization:
  - Input: Image
  - Output: Box in the image (x, y, w, h)
  - Loss: L2 Loss (Euclidean distance)
  - Evaluation metric: Intersection over Union
- Classification + Localization:
  - Input: Image
  - Output: Class label + box in the image
  - Loss: Sum of both losses







# ImageNet Challenge



- Dataset
  - 1000 Classes.
  - Each image has 1 class with at least one bounding box.
  - ~800 Training images per class.
- Evaluation
  - Algorithm produces 5 (class + bounding box) guesses.
  - Example is correct if at least one of guess has correct class AND bounding box at least 50% intersection over union.

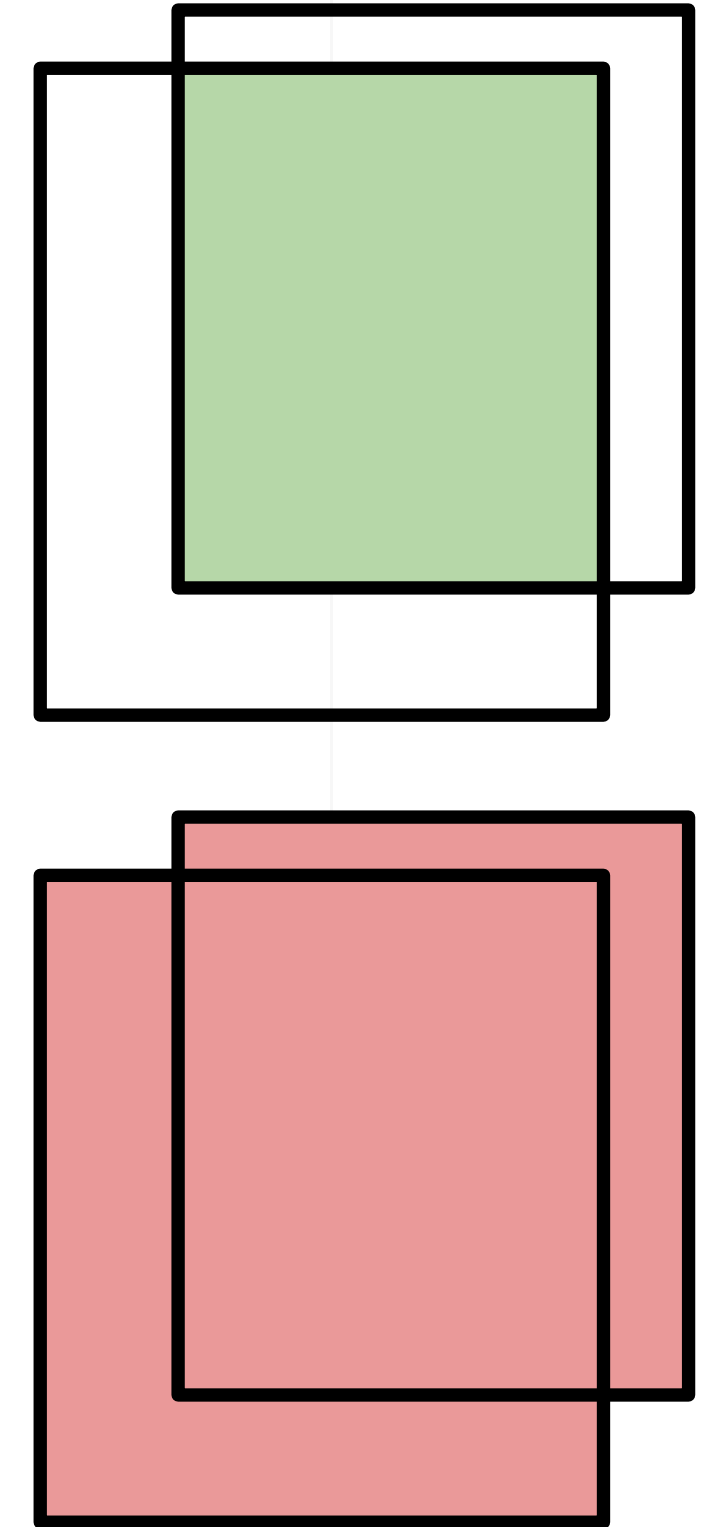






# Intersection Over Union (IoU)

$$\text{IoU}(A,B) = \frac{\text{Intersection}(A,B)}{\text{Union}(A,B)}$$



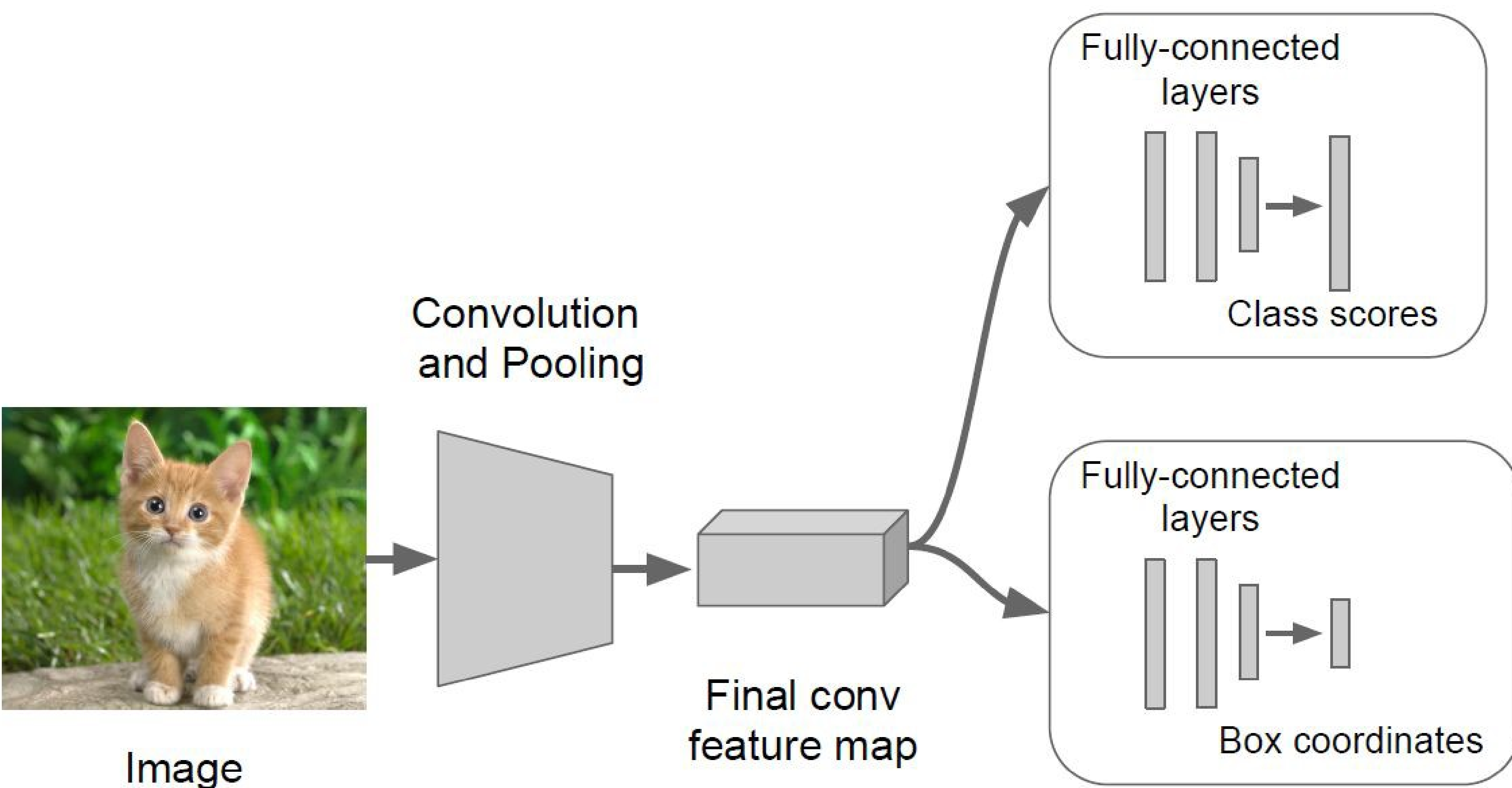
21 October  
2025

- Important ' measurement for object localization.
- Used in both training and evaluation.





# Classification + Localization: Model



## Classification Head:

- C Scores for C classes

## Localization Head:

- Class agnostic:  $(x, y, w, h)$
- Class specific:  $(x, y, w, h) \times C$



# Computer Vision Tasks

Classification



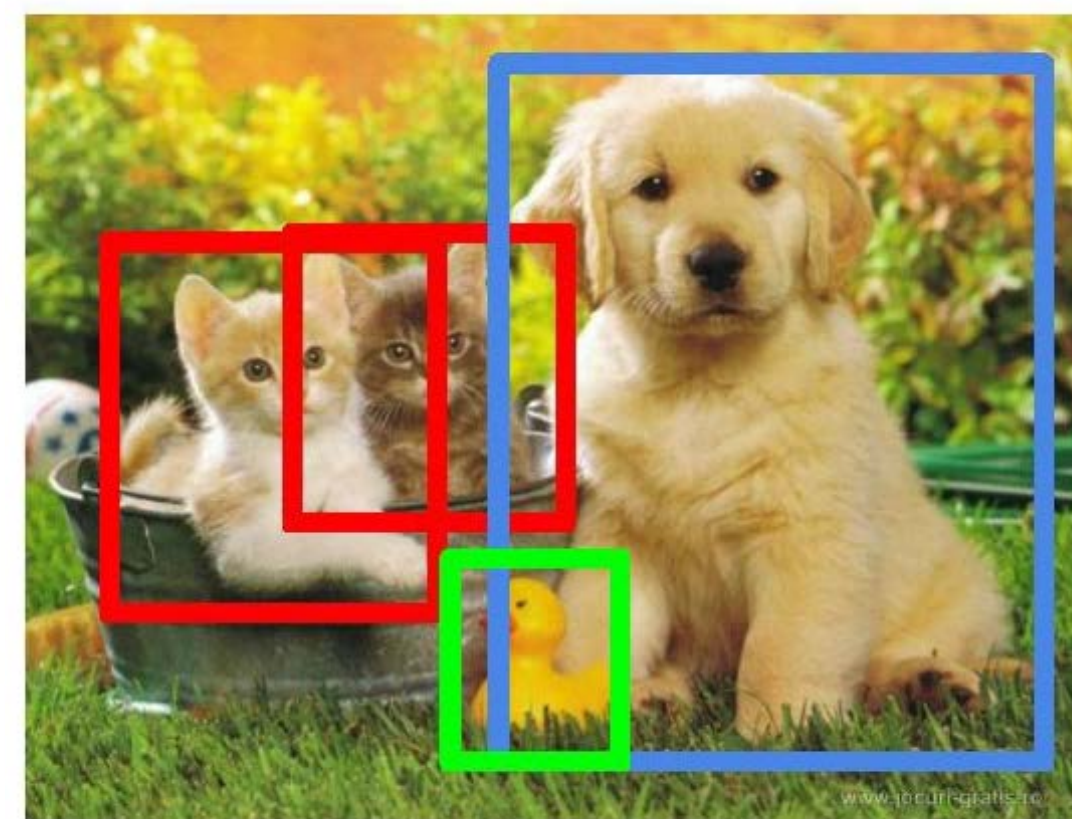
CAT

Classification  
+ Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance  
Segmentation



CAT, DOG, DUCK

Single object

Multiple objects

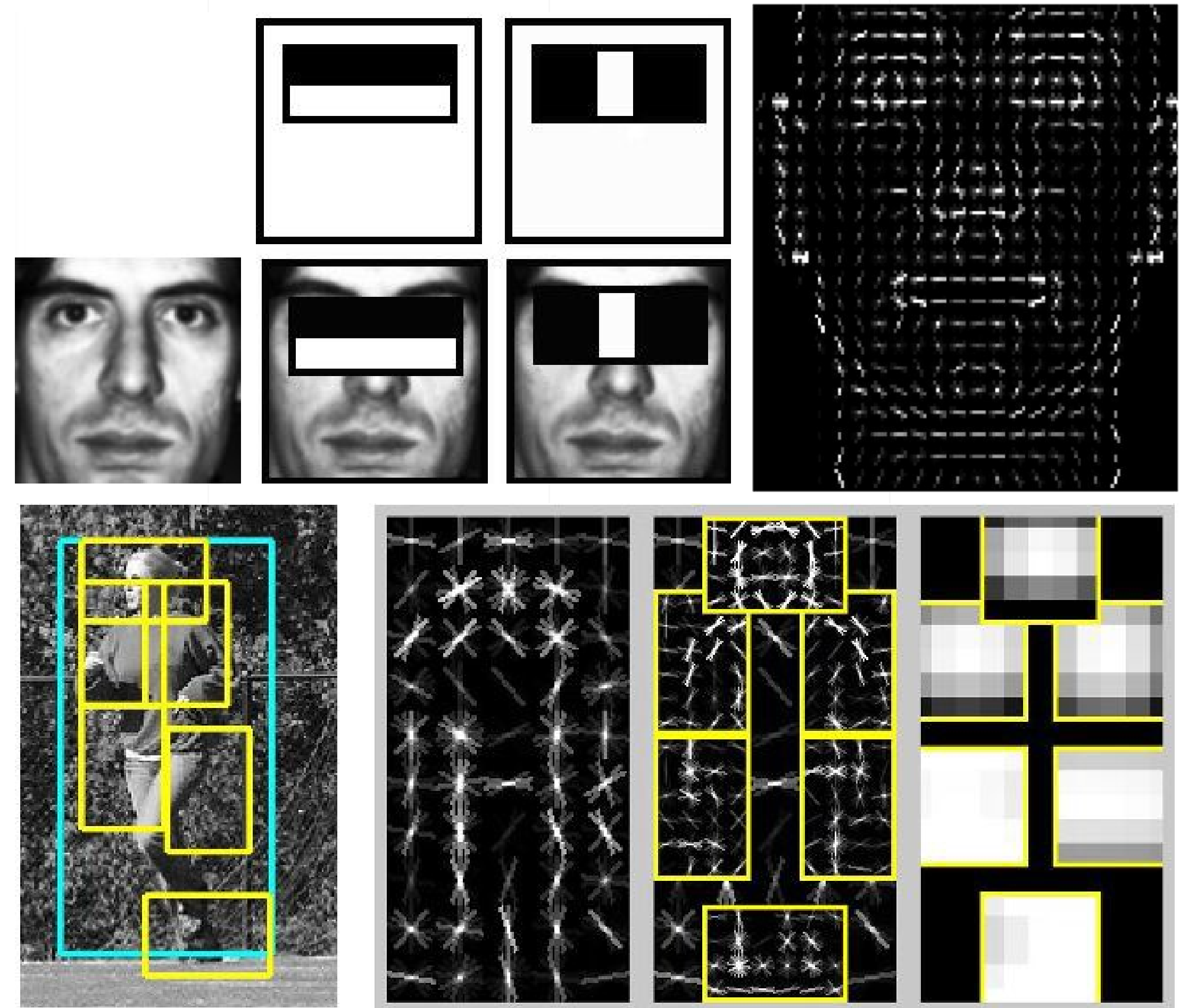




# Object Detection 2001-2007

- Rapid Object Detection using a Boosted Cascade of Simple Features (2001)
  - Viola & Jones
- Histograms of Oriented Gradients for Human Detection (2005)
  - Dalal & Triggs
- Object Detection with Discriminatively Trained Part Based Models (2010)
  - Felzenszwalb, Girshick, Ramanan
- Fast Feature Pyramids for Object Detection (2014)
  - Dollar

21 October  
2025





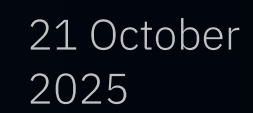
# Object Detection 2001-2007



21 October  
2025











# Object Detection Datasets

2007

## Pascal VOC

- 20 Classes
- 11K Training images
- 27K Training objects

Was de-facto standard, currently used as quick benchmark to evaluate new detection algorithms.

2013

## ImageNet ILSVRC

- 200 Classes
- 476K Training images
- 534K Training objects

Essentially scaled up version of PASCAL VOC, similar object statistics.

2015

## MS COCO

- 80 Classes
- 328k Training images
- 1.5M Training objects

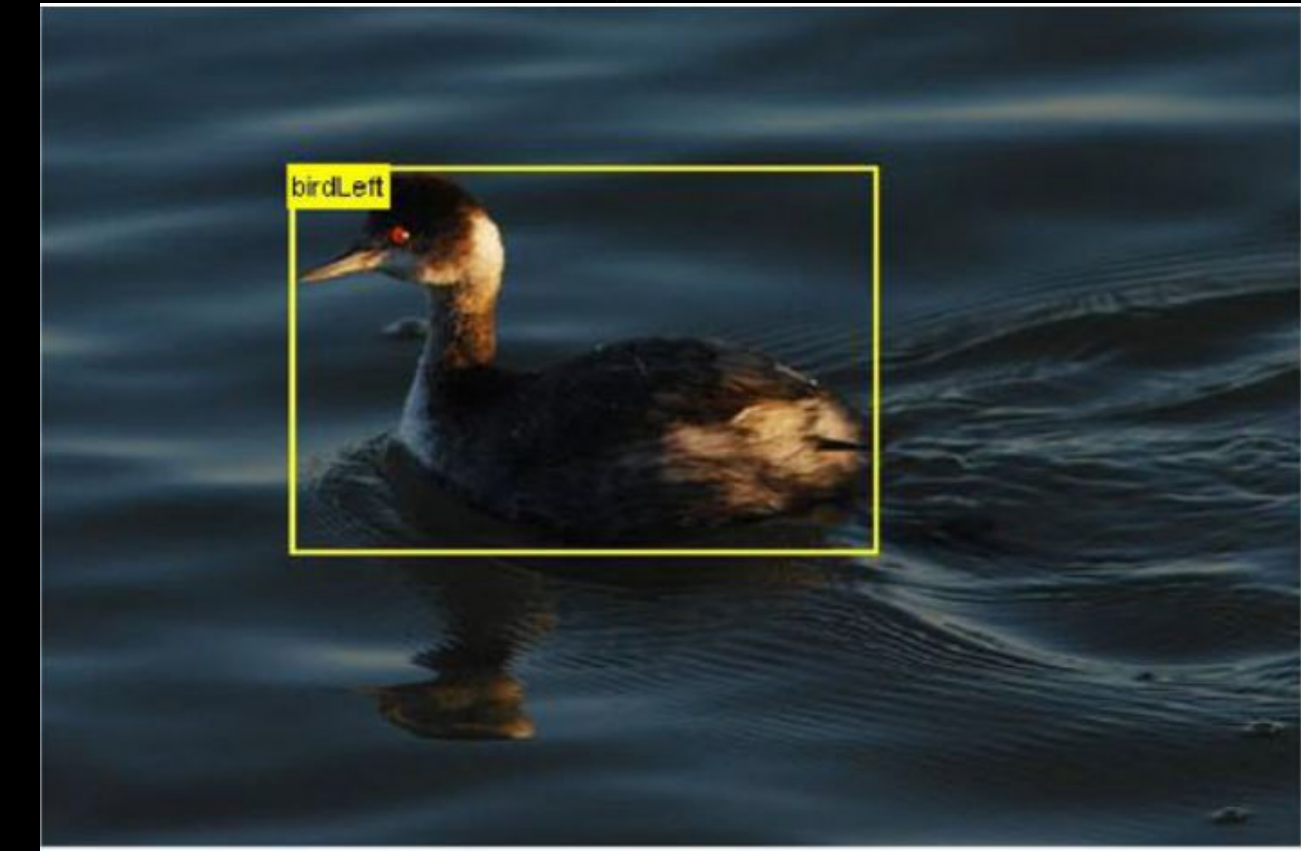
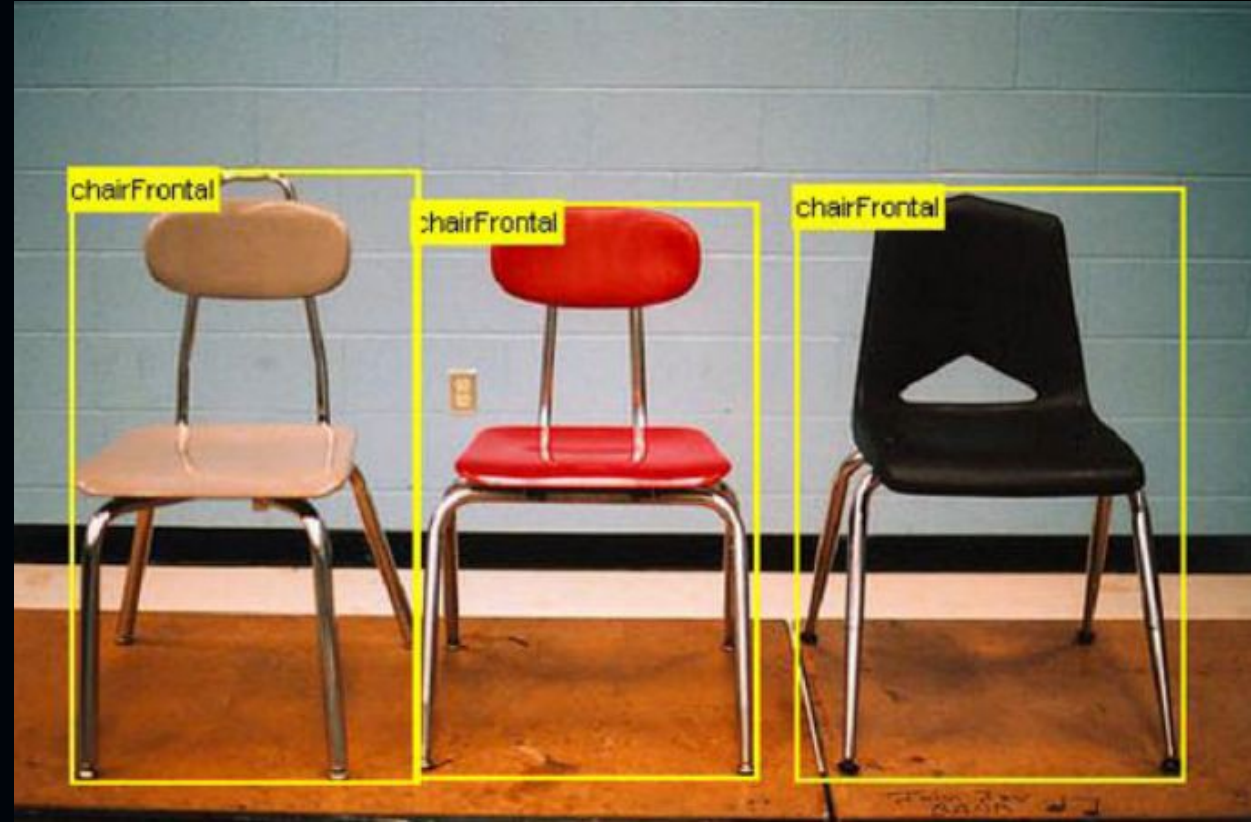
More categories and more object instances in every image. Only 10% of images contain a single object category, 60% in Pascal. More small objects than large objects.







# Pascal Examples



21 October  
2025







# COCO Examples



21 October  
2025



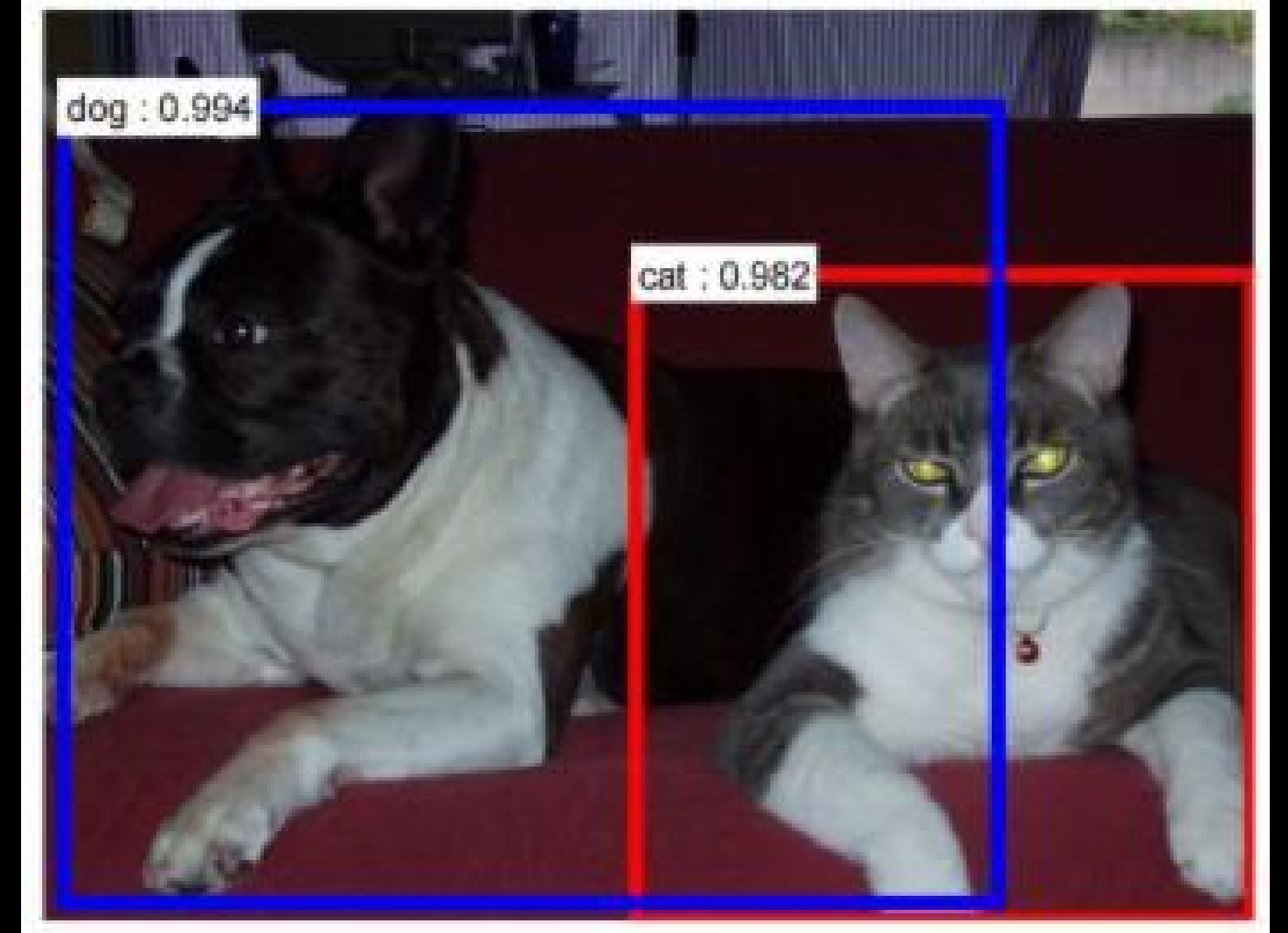
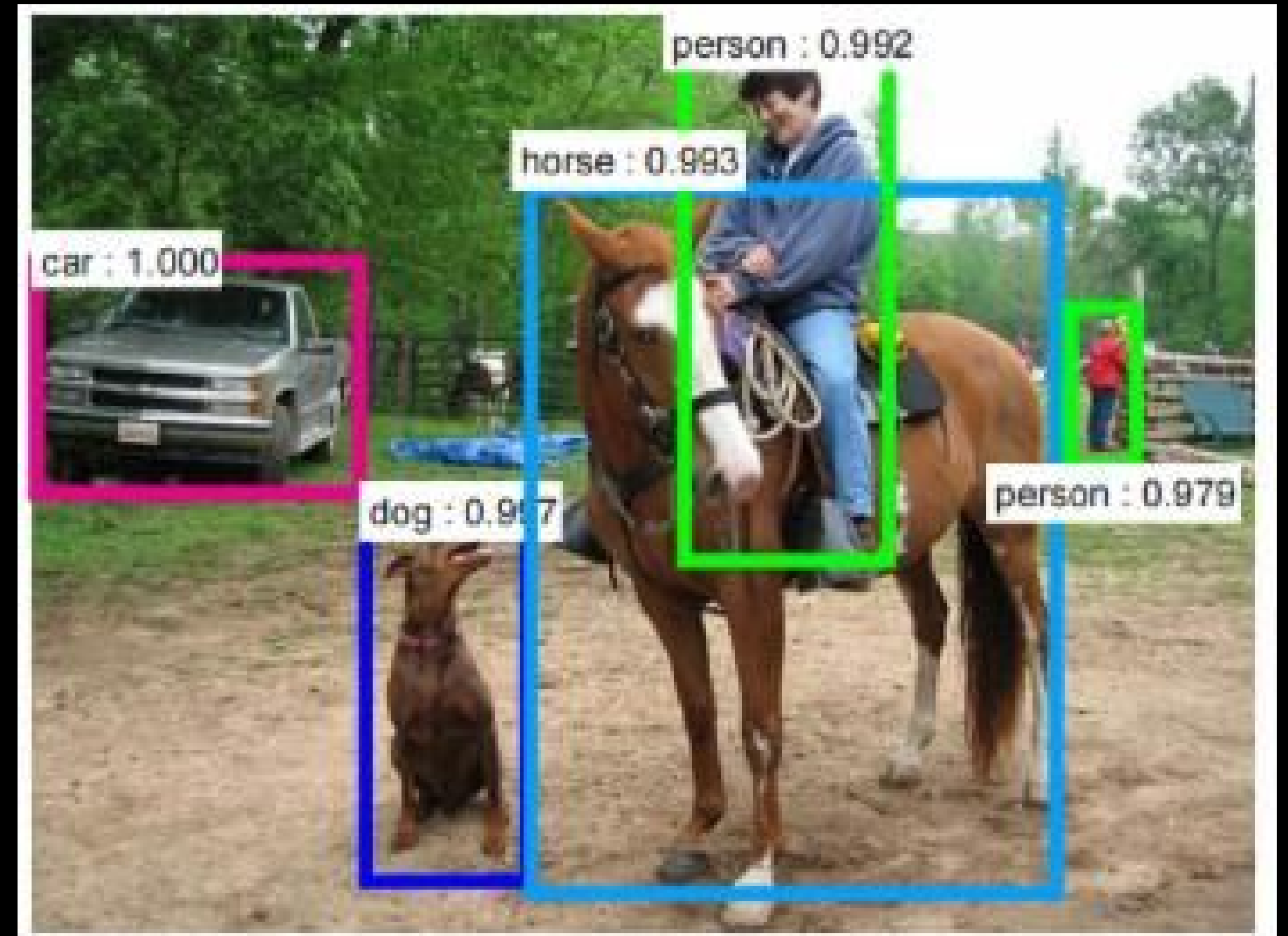




# Object Detection

- Input: Image
- Output: For each object class  $c$  and each image  $i$ , an algorithm returns predicted detections:  $\{(b_{ij}, s_{ij})\}_{j=1}^M$  locations.  $b_{ij}$  with confidence scores  $s_{ij}$ .

21 October  
2025

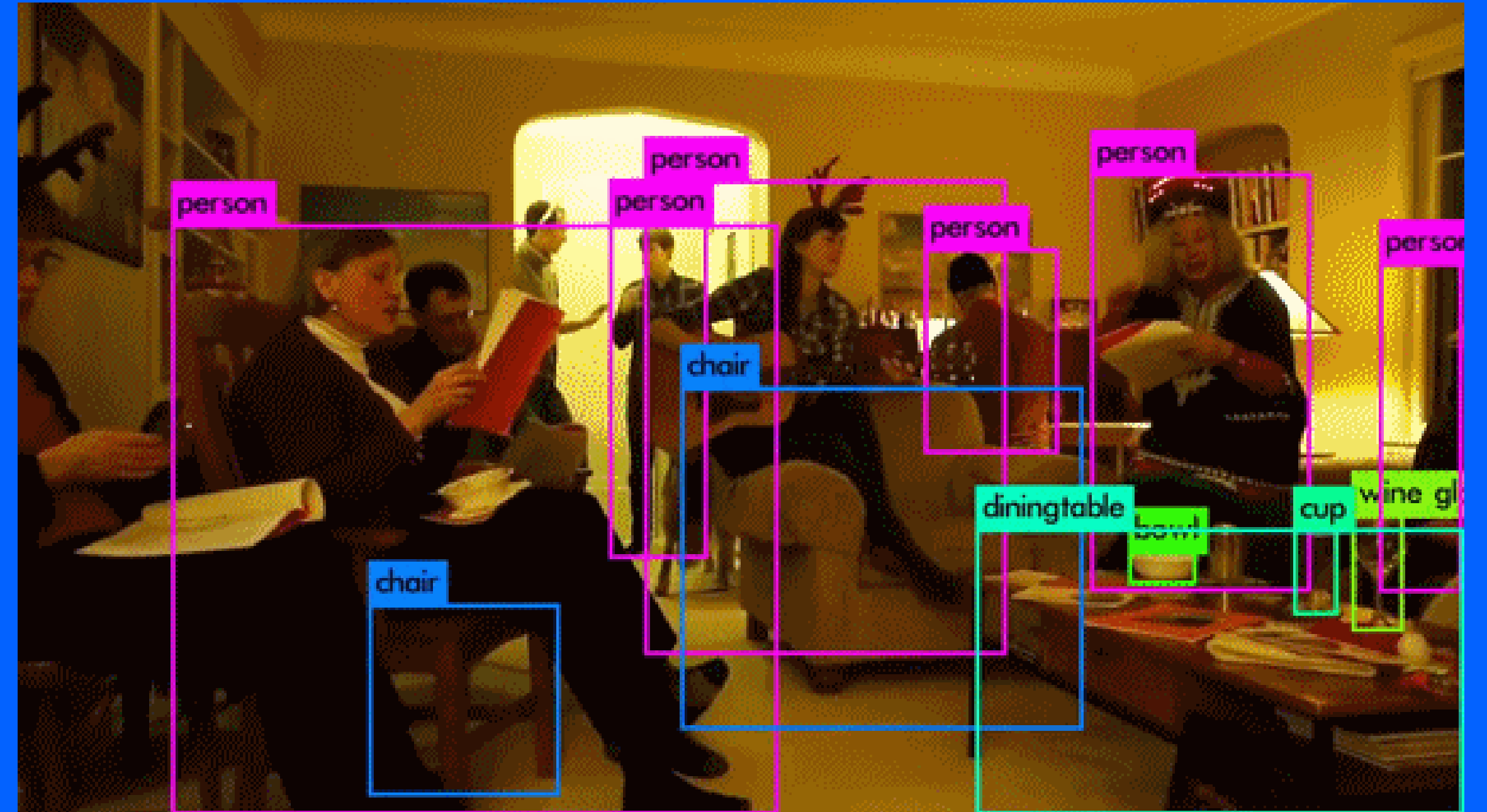




# YOLO (You Only Look Once)

Unified Real-Time Object Detection

21 October  
2025



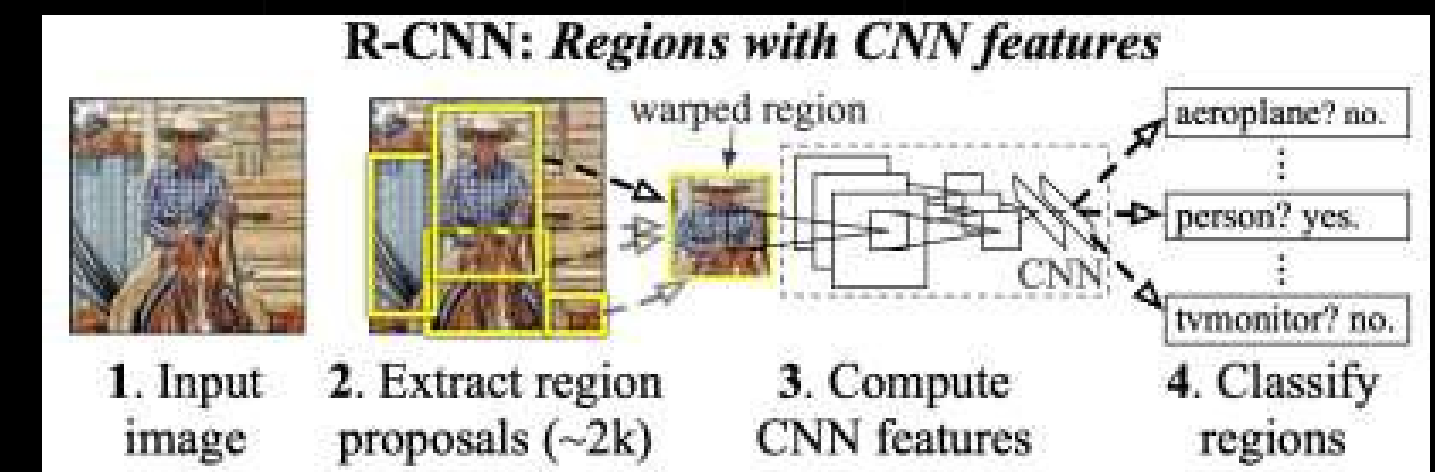
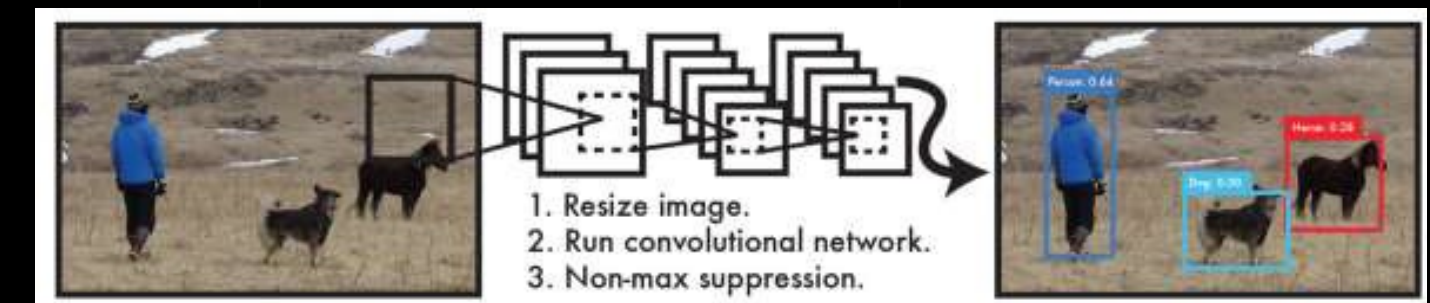
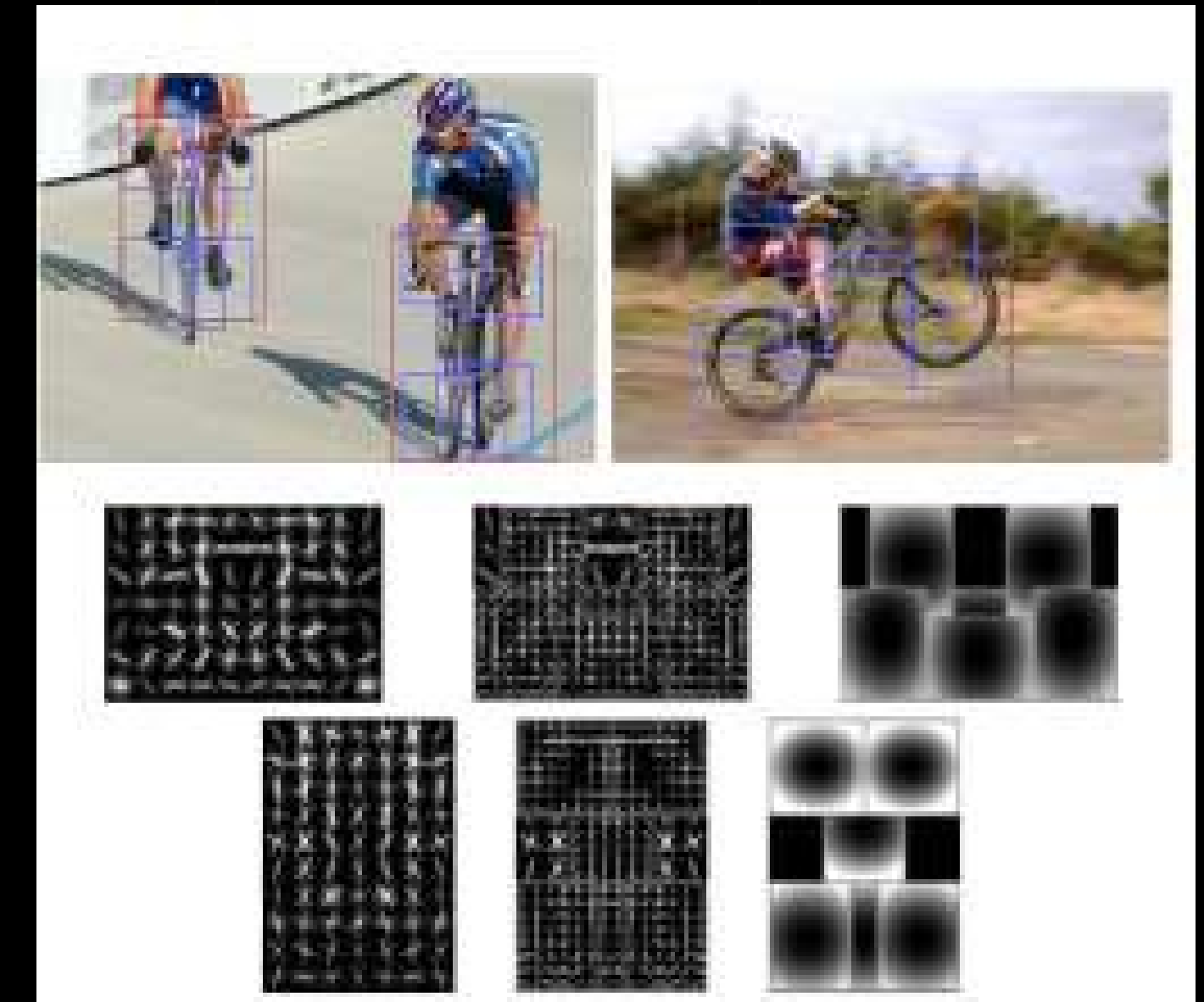




# Previously: Object Detection with Classifiers

- DPM (Deformable Parts Model)
  - Sliding window → classifier (evenly spaced locations)
- R-CNN
  - Region proposal --> potential BB
  - Run classifiers on BB
  - Post-processing (refinement, eliminate, rescore)
- YOLO
  - Resize image, run the convolutional network, non-max suppression

21 October  
2025





# YOLO : Object Detection as Regression Problem

Output: Bounding box coordinates and Class Probabilities

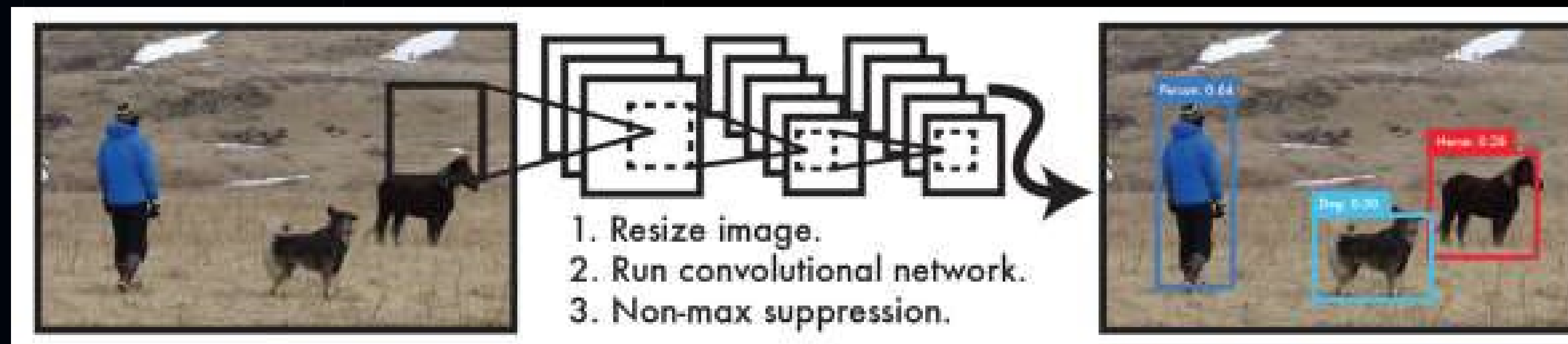
Single Neural Network

Benefits:

Extremely Fast (one NN + 45 frames per sec),

Global Reasoning (knows the context, fewer background errors)

Generalizable Representations (train natural images, test artwork, applicable new domain)







# Unified Detection

- Feature Extraction
  - Predict all class BB simultaneously
- SxS Grid
  - Each cell predicts B bounding boxes + Confidence Score
- Confidence Score
  - Confidence is IOU between predicted box and any ground truth box =
- Class Probability
- Tensor

confidence as  $\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$

At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

$\text{IOU}_{\text{pred}}^{\text{truth}}$

$S \times S \times (B * 5 + C)$  tensor.

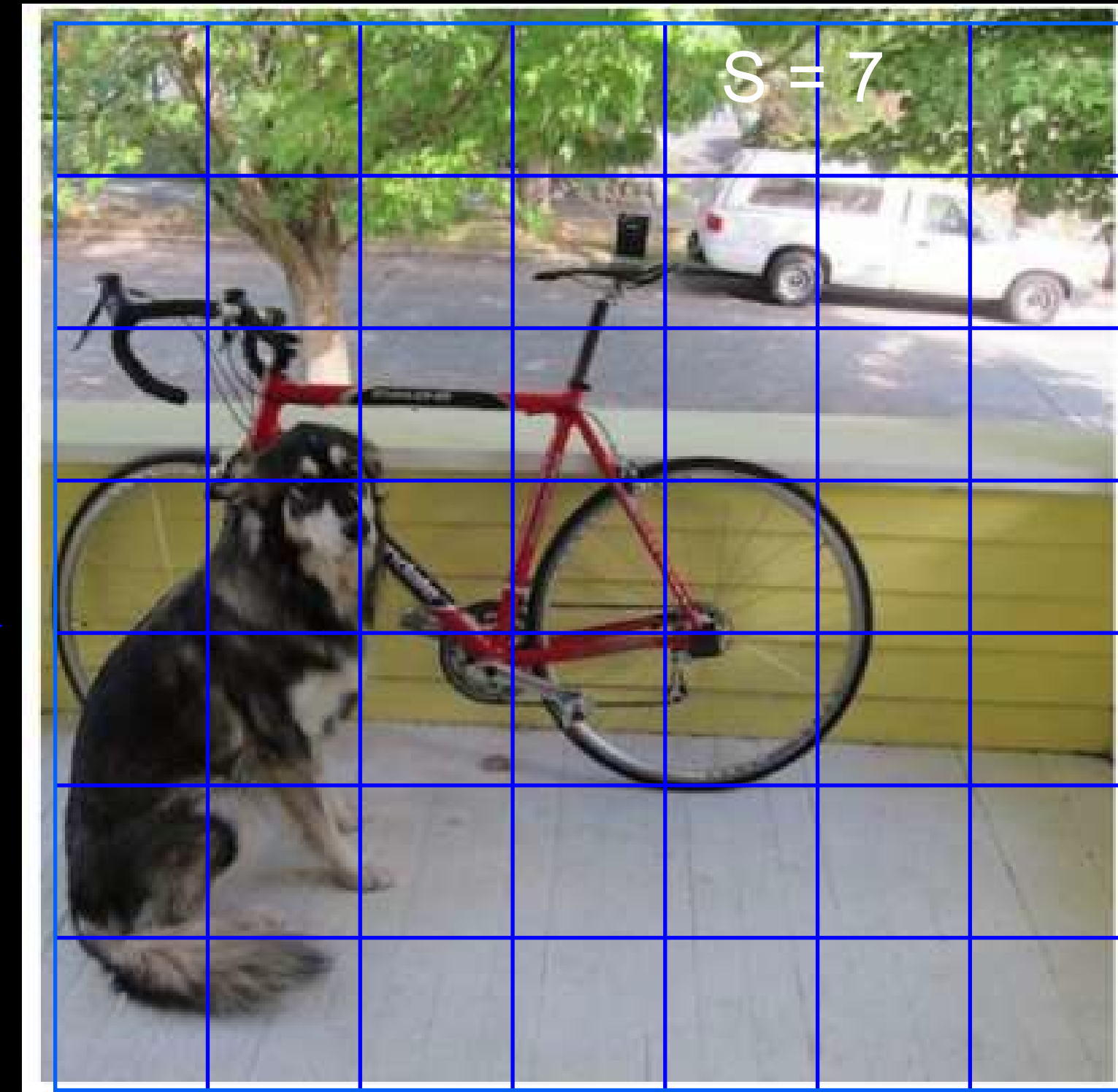
For evaluating YOLO on PASCAL VOC, we use  $S = 7$ ,  $B = 2$ . PASCAL VOC has 20 labelled classes so  $C = 20$ . Our final prediction is a  $7 \times 7 \times 30$  tensor.





# Detection Process (YOLO)

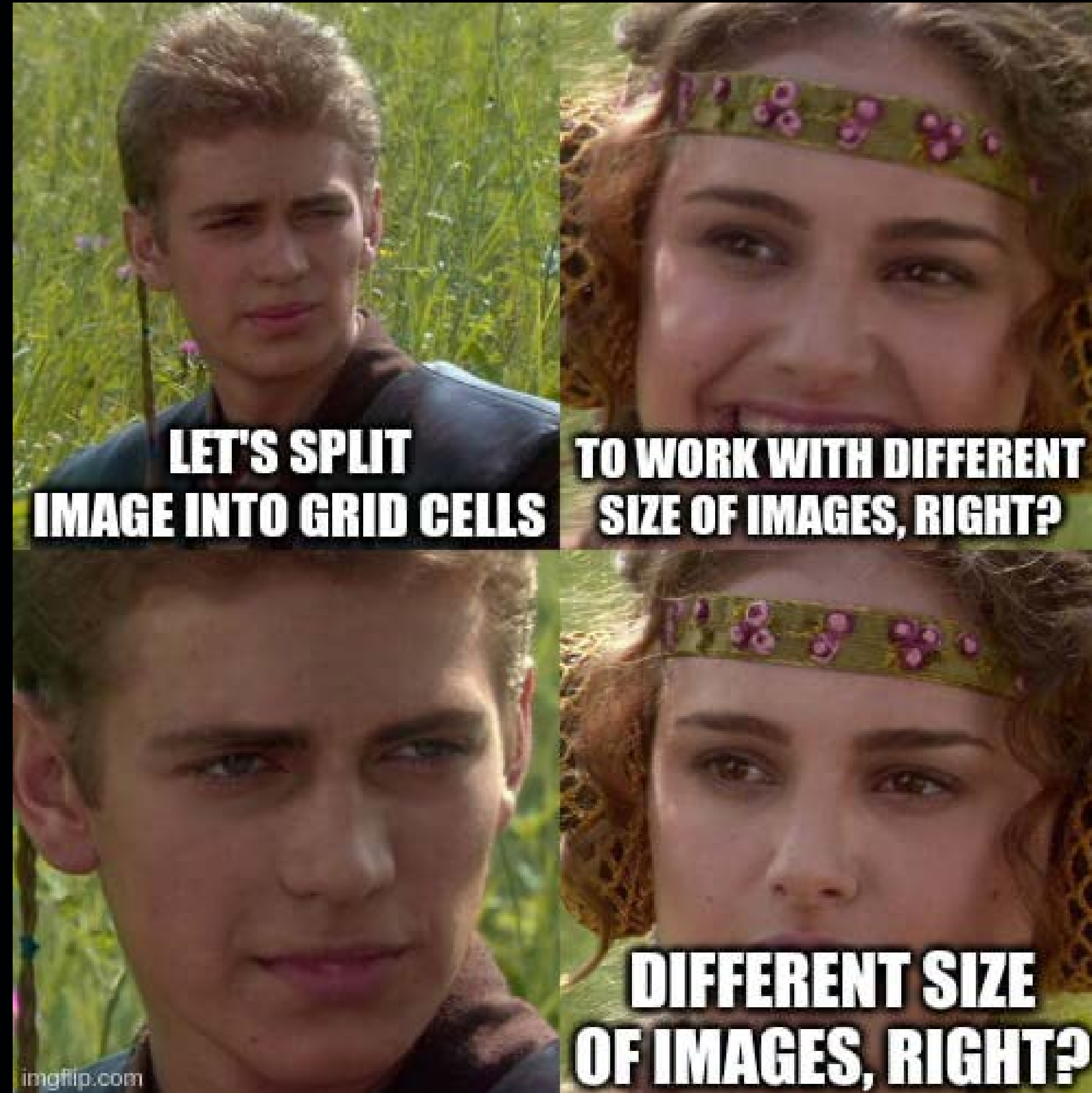
Grid SXS







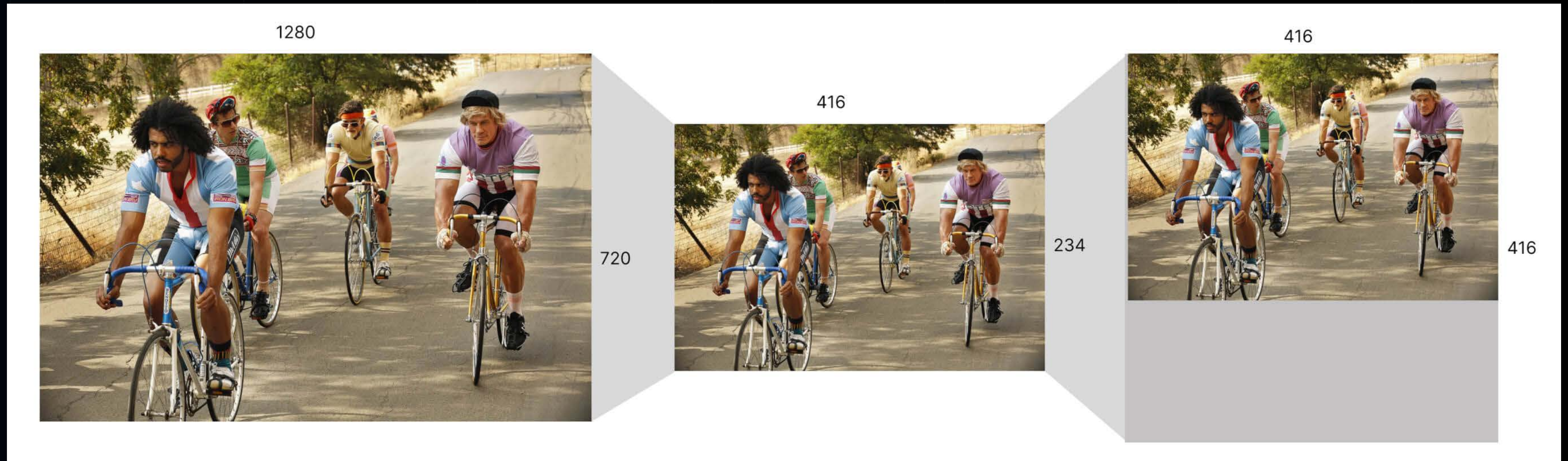
21 October  
2025







# Original YOLO implementation resizes the image to a specific size to preserve the Grid Structure.



Neural Networks require fixed size inputs to streamline the computation and optimize performance

Aspect Ratio is preserved!







# Confidence score

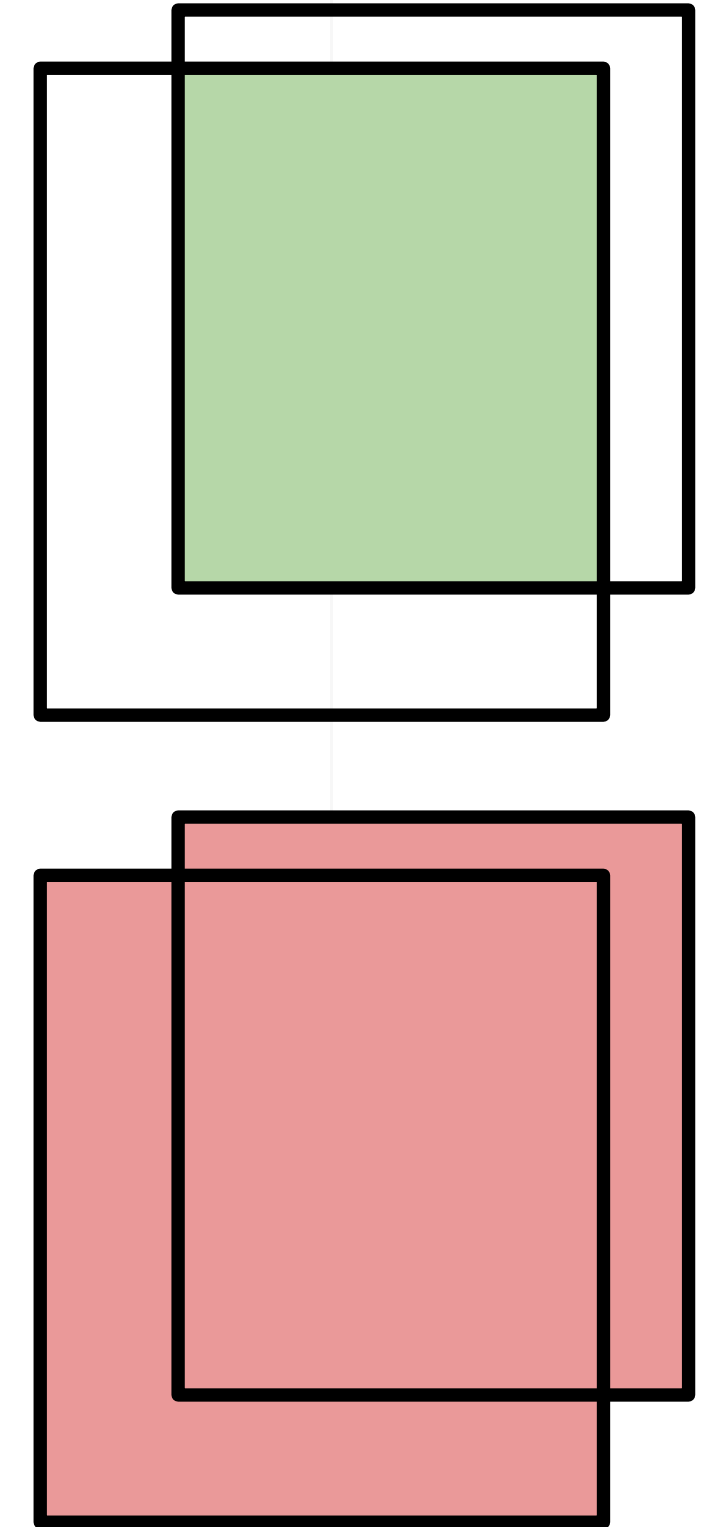
- Each grid cell predicts B bounding boxes and confidence scores for those boxes.
- If a cell has an object, then confidence score = Intersection over union (IOU) between the predicted box and the ground truth.

21 October  
2025



$$\text{IoU}(A,B) = \frac{\text{Intersection}(A,B)}{\text{Union}(A,B)}$$

$$\text{Confidence} = \text{Pr}(\text{Object}) * \text{IoU}$$



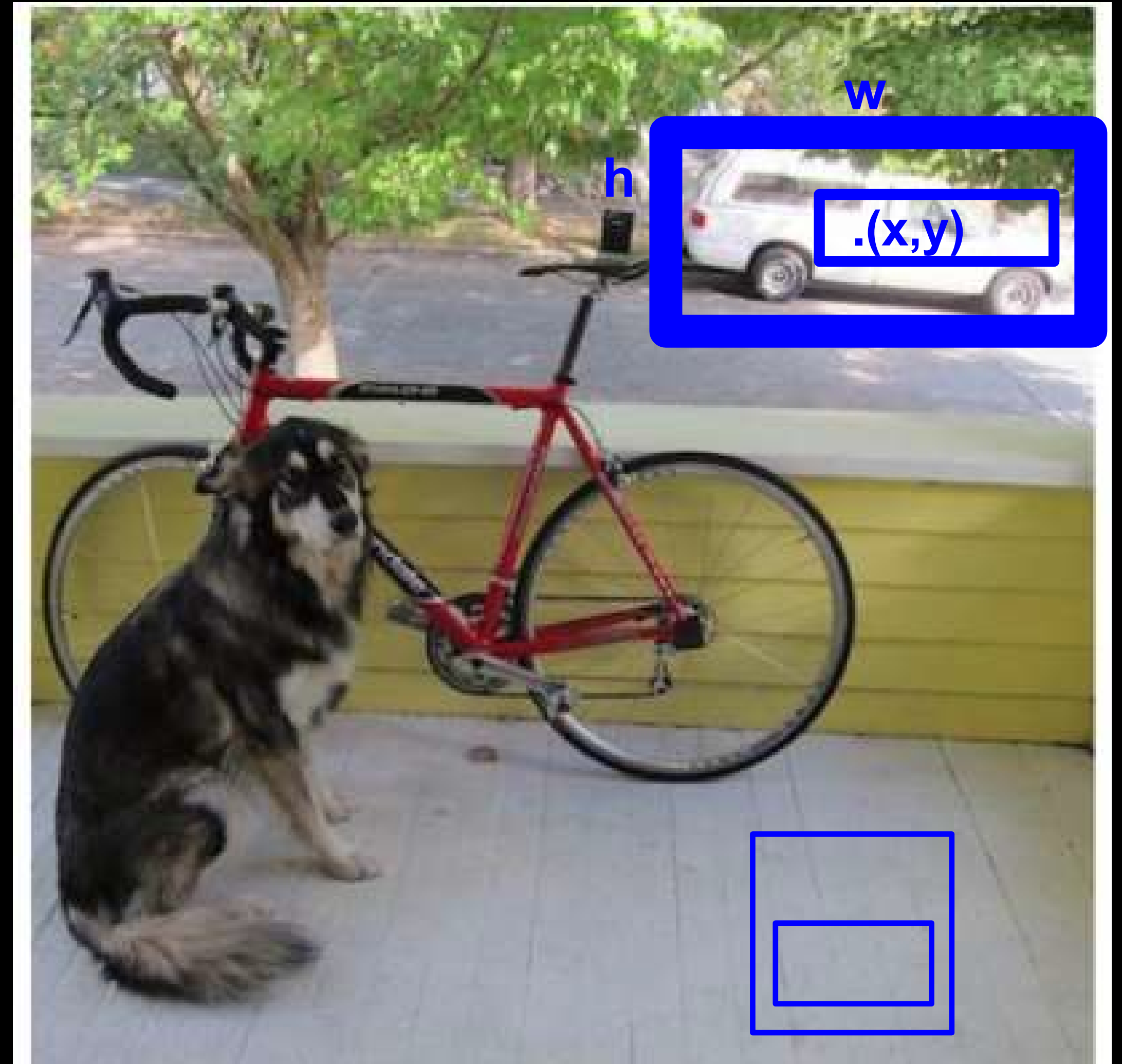
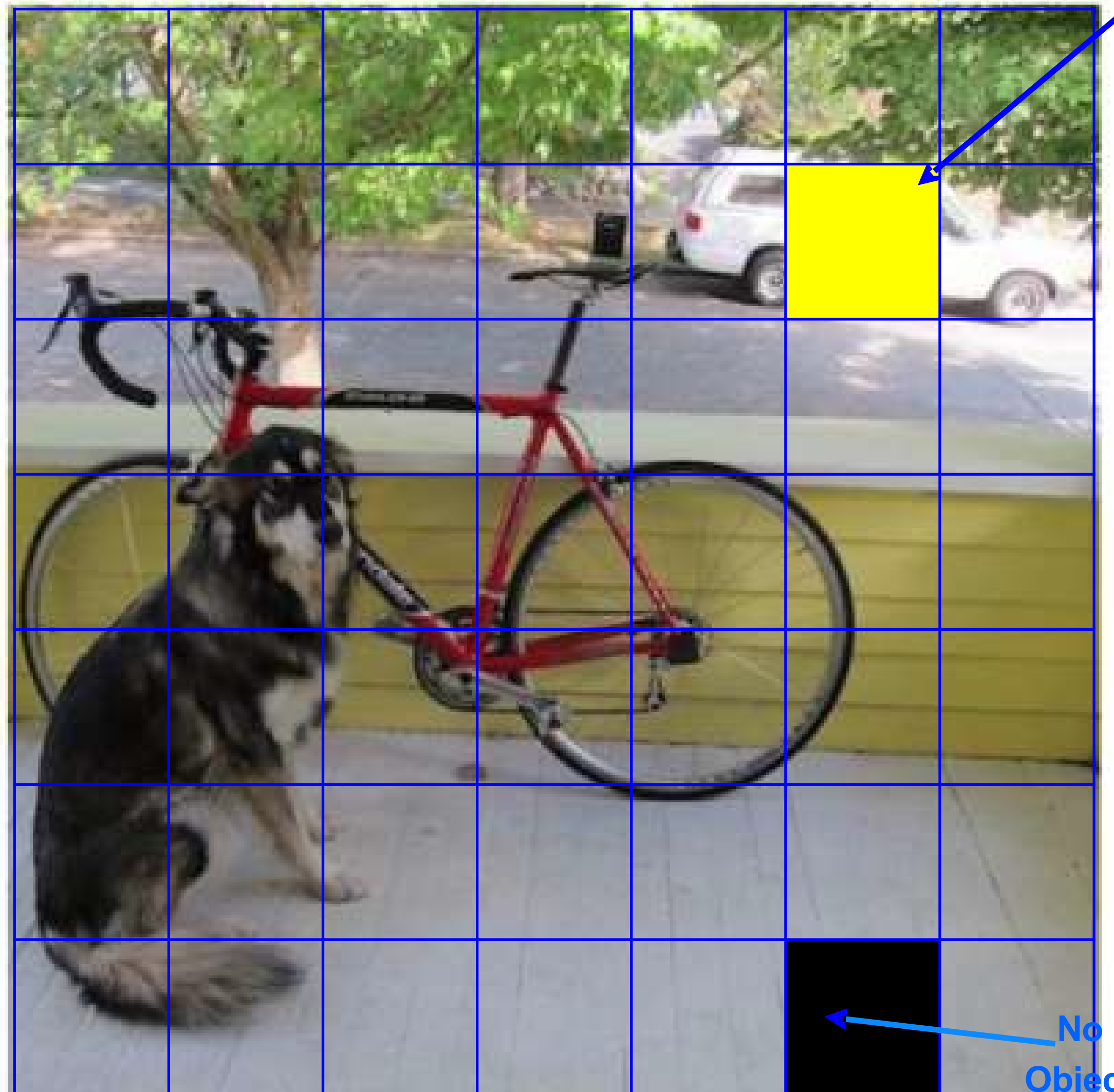


# Detection Process (YOLO)

Each cell predicts  $B$  boxes  $(x,y,w,h)$  and confidences of each box:  $P(\text{Object})$

Prob. that box contains an object  $P1, P2$

$B = 2$



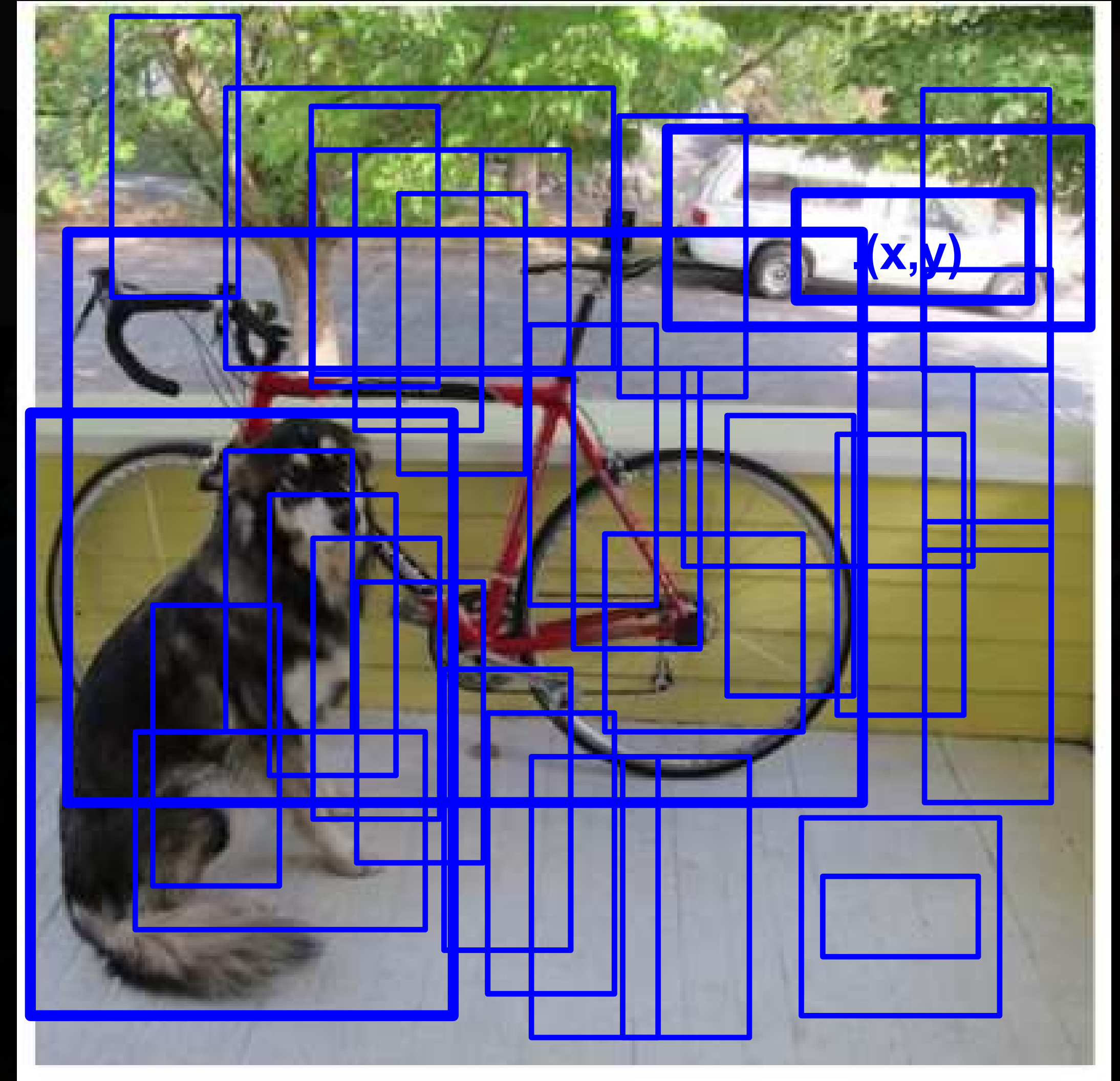
21 October  
2025







# Each cell predicts Bounding Boxes and Confidence

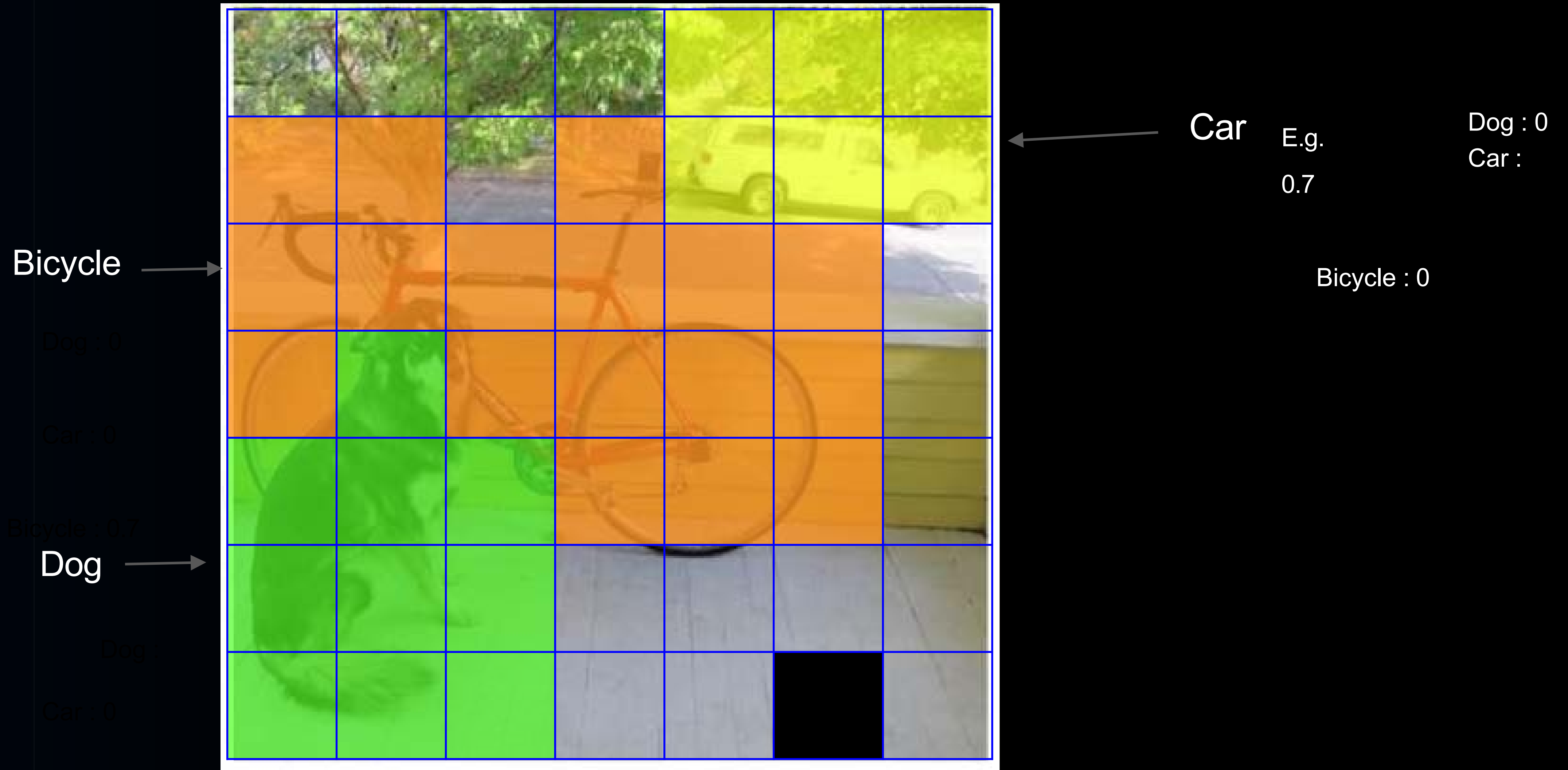


21 October  
2025





# Each cell also predicts class probability



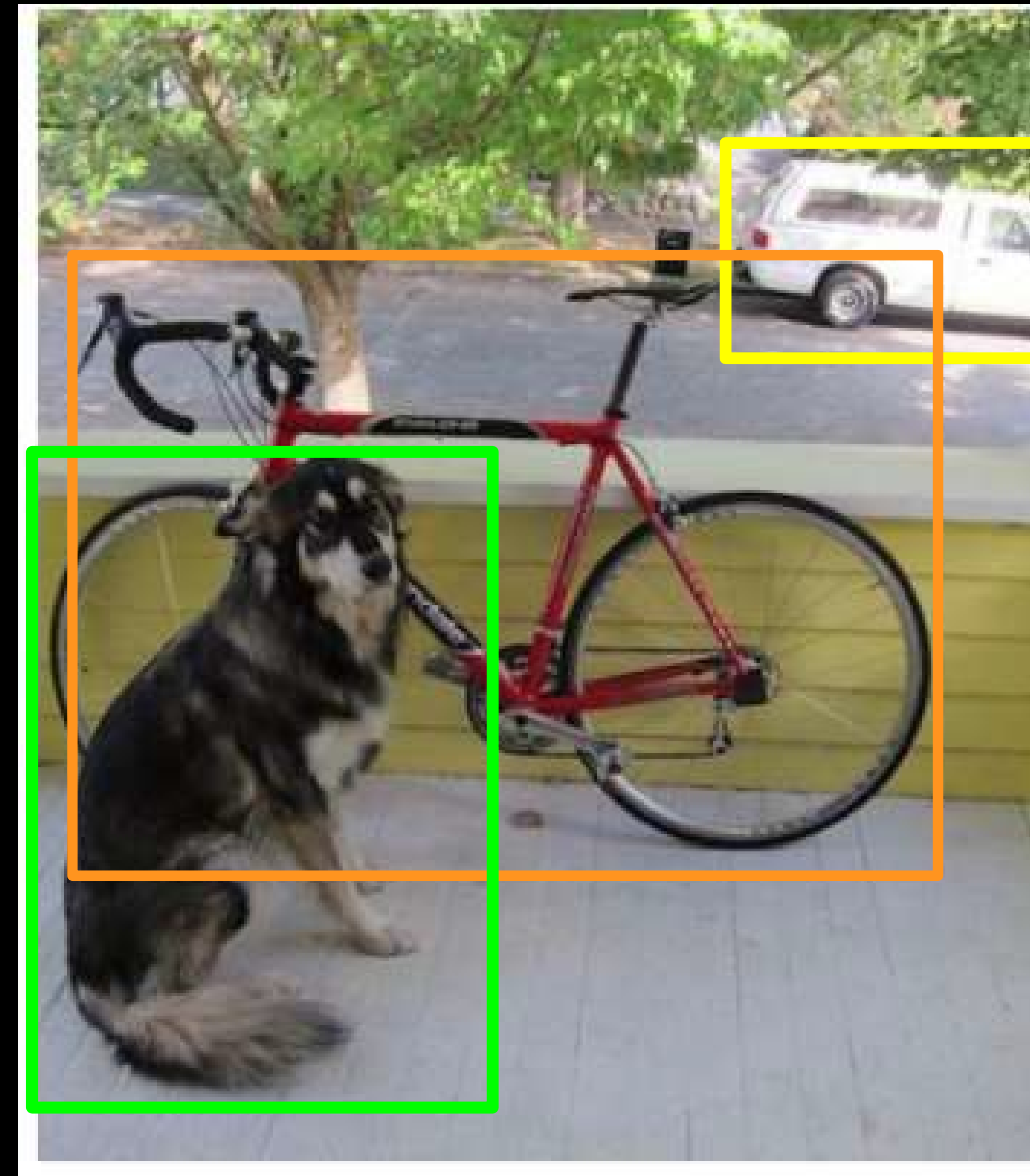
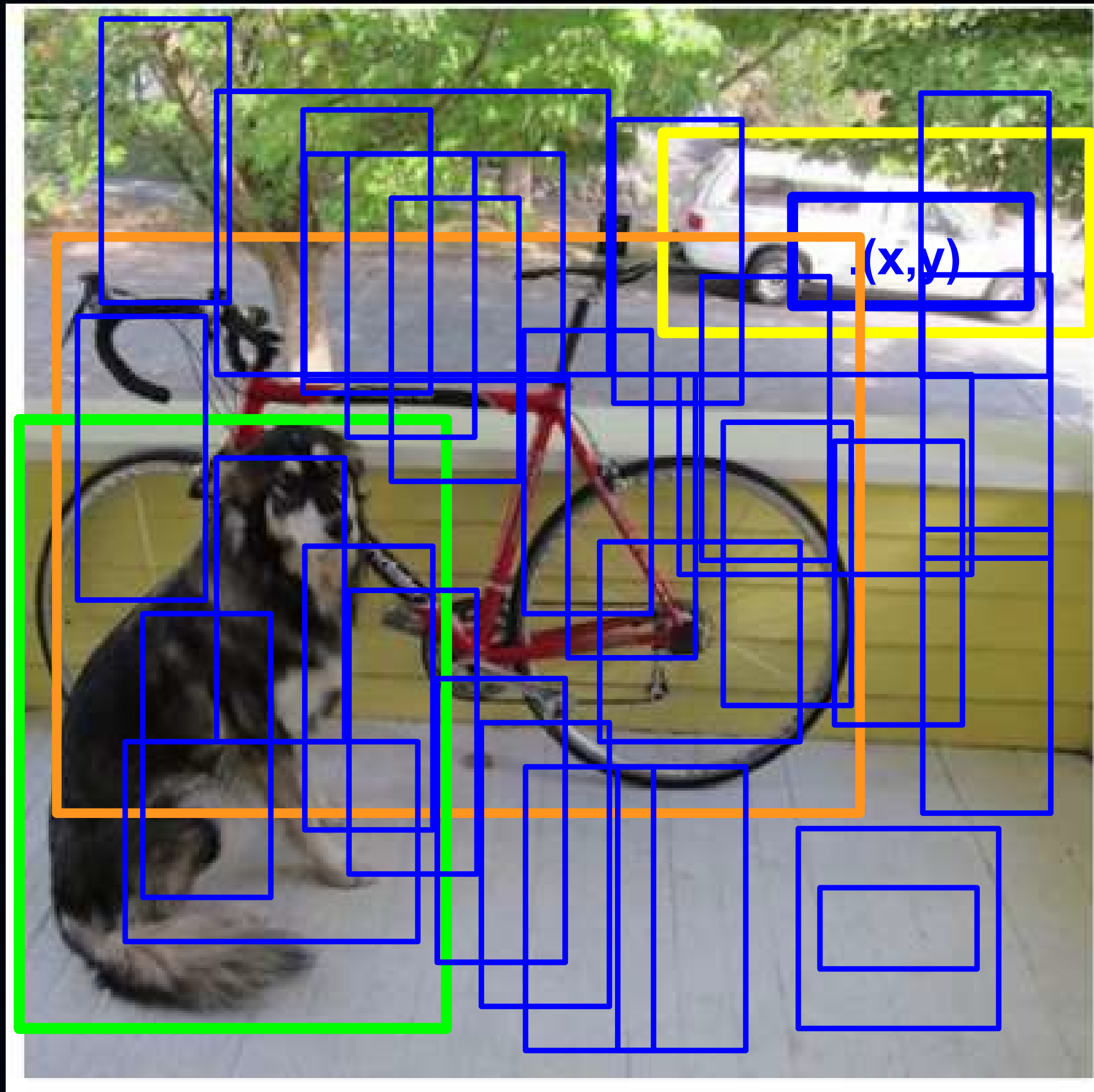




# Bounding Boxes + Class Prediction

$$P(\text{class}) = P(\text{class}|\text{object}) \times P(\text{object})$$

Thresholding



E.g.

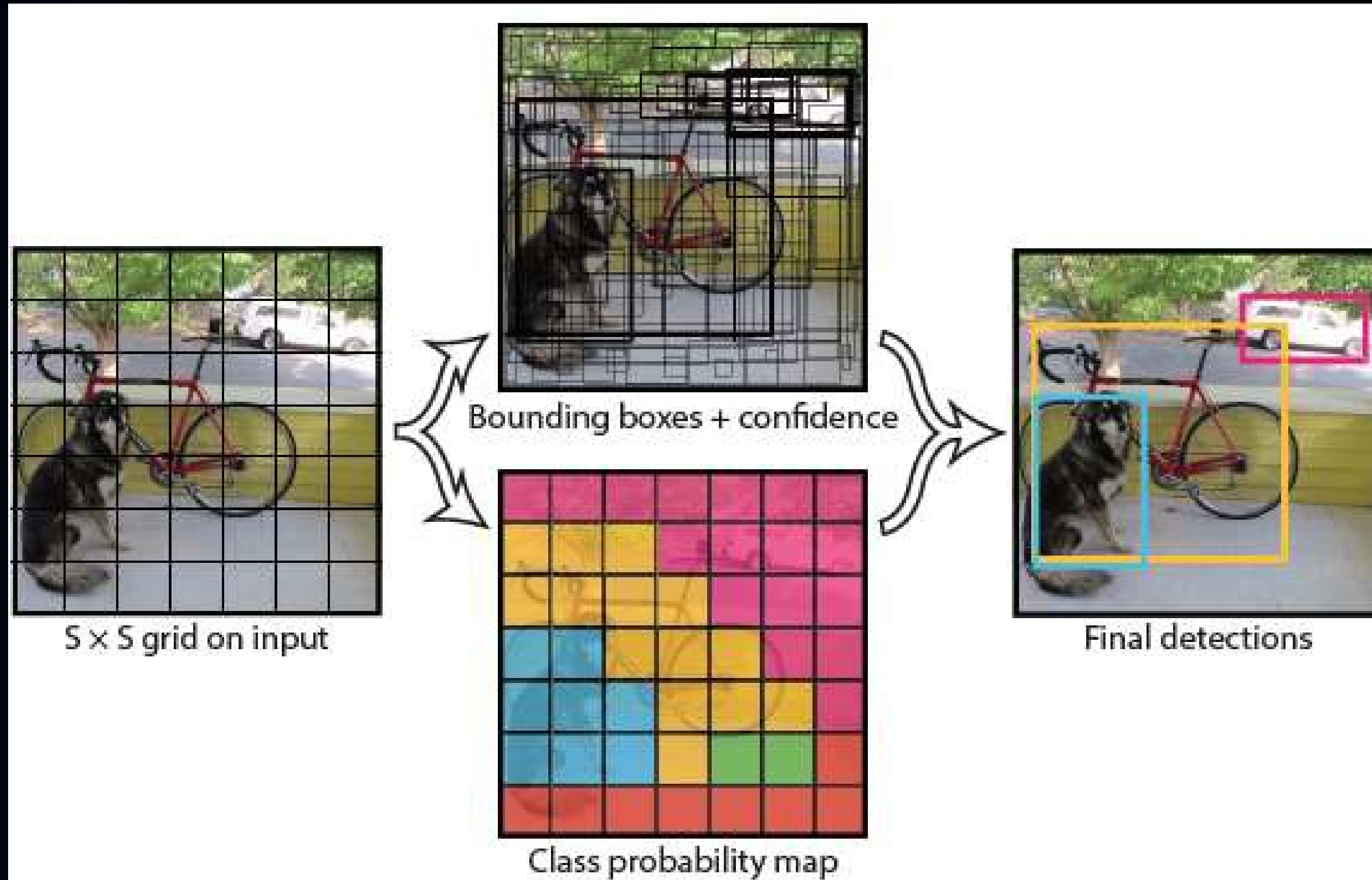
21 October  
2025

E.g.  
0.8





# Model



These predictions are encoded as Tensor of dimension  $(S \times S \times (B \times 5 + C))$   
 $S \times S$  grid,  
 $C$  = class probability,  
 $B$  = no of bounding boxes.

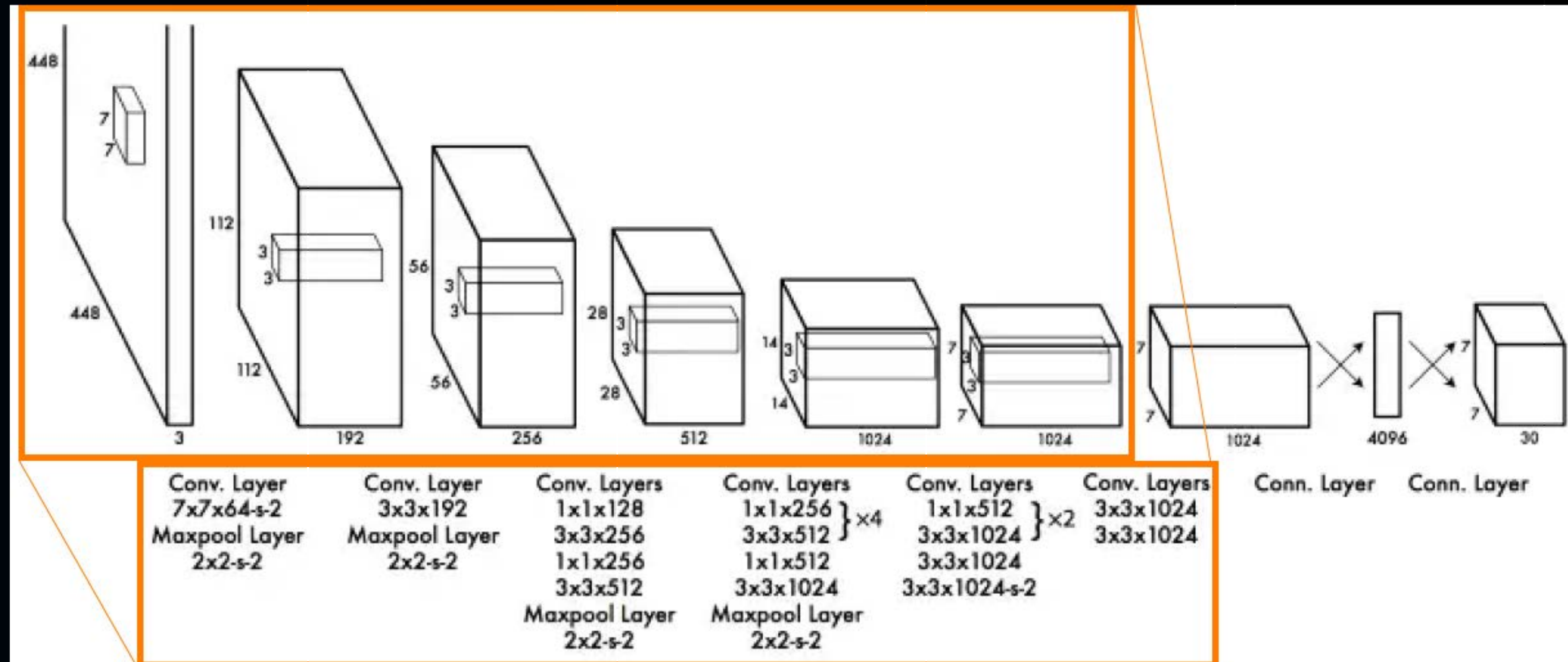






# Network Design

- Inspired by the GoogLeNet (image classification)
- 24 convolutional layers followed by 2 fully connected layers
- Fast YOLO uses 9 convolutional layers (instead of 24)



First of all, 24 convolutional layers that "stretch" an image with size (448x448x3: w,h, RGB channels) into a tensor with the size of (SxSx1024, where S=7).





# Loss Function

## 1. What is a Loss Function?

- A tool to measure how well our model is performing.
- A lower loss indicates better performance, meaning our predictions are close to the actual values.

## 2. Components of YOLO Loss Function:

- Localization Loss
- Classification Loss
- Confidence Loss

## 3. Balancing the Loss:

- Weights are added to balance the contribution of different components in the loss function.
- For example, YOLO assigns more importance to accurate localization than accurate classification.

## 4. The objective of Training:

- The goal is to adjust the model parameters to minimize the loss function.
- A more minor loss means the model's predictions are closer to the truth.

$$\begin{aligned} & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 \\ & \quad + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \\ & \quad \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$







# Loss Function

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[ (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[ \left( \sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left( \sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3) \end{aligned}$$

where  $\mathbb{1}_i^{\text{obj}}$  denotes if object appears in cell  $i$  and  $\mathbb{1}_{ij}^{\text{obj}}$  denotes that the  $j$ th bounding box predictor in cell  $i$  is “responsible” for that prediction.

model. We use sum-squared error because it is easy to optimize, however it does not perfectly align with our goal of maximizing average precision. It weights localization error equally with classification error which may not be ideal. Also, in every image many grid cells do not contain any object. This pushes the “confidence” scores of those cells towards zero, often overpowering the gradient from cells that do contain objects. This can lead to model instability, causing training to diverge early on.

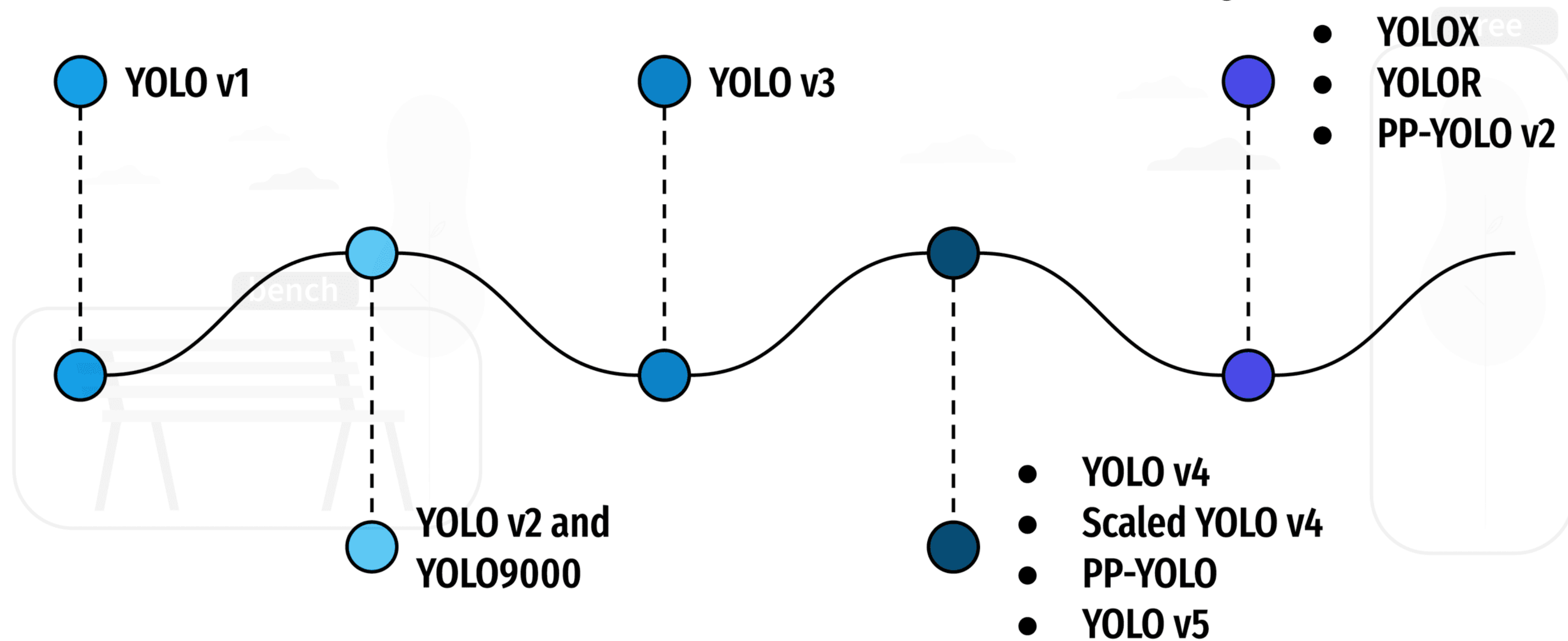
To remedy this, we increase the loss from bounding box coordinate predictions and decrease the loss from confidence predictions for boxes that don't contain objects. We use two parameters,  $\lambda_{\text{coord}}$  and  $\lambda_{\text{noobj}}$  to accomplish this. We set  $\lambda_{\text{coord}} = 5$  and  $\lambda_{\text{noobj}} = .5$ .

Sum-squared error also equally weights errors in large boxes and small boxes. Our error metric should reflect that small deviations in large boxes matter less than in small boxes. To partially address this we predict the square root of the bounding box width and height instead of the width and height directly.





# Introduction to the YOLO Family







# Positives & Limitations of YOLO

## Pro:

- Accurate predictions for real-time object detection models using natural images.
- Good inference speed compared to some state-of-the-art (for that time) object detection architectures.
- Impressive generalizability: when R-CNN algorithms have a massive drop in average precision within the domain shift, the YOLO algorithm's average precision drops slightly.
- Fewer background errors compared to the Fast R-CNN algorithm (4.75% vs. 13.6%).

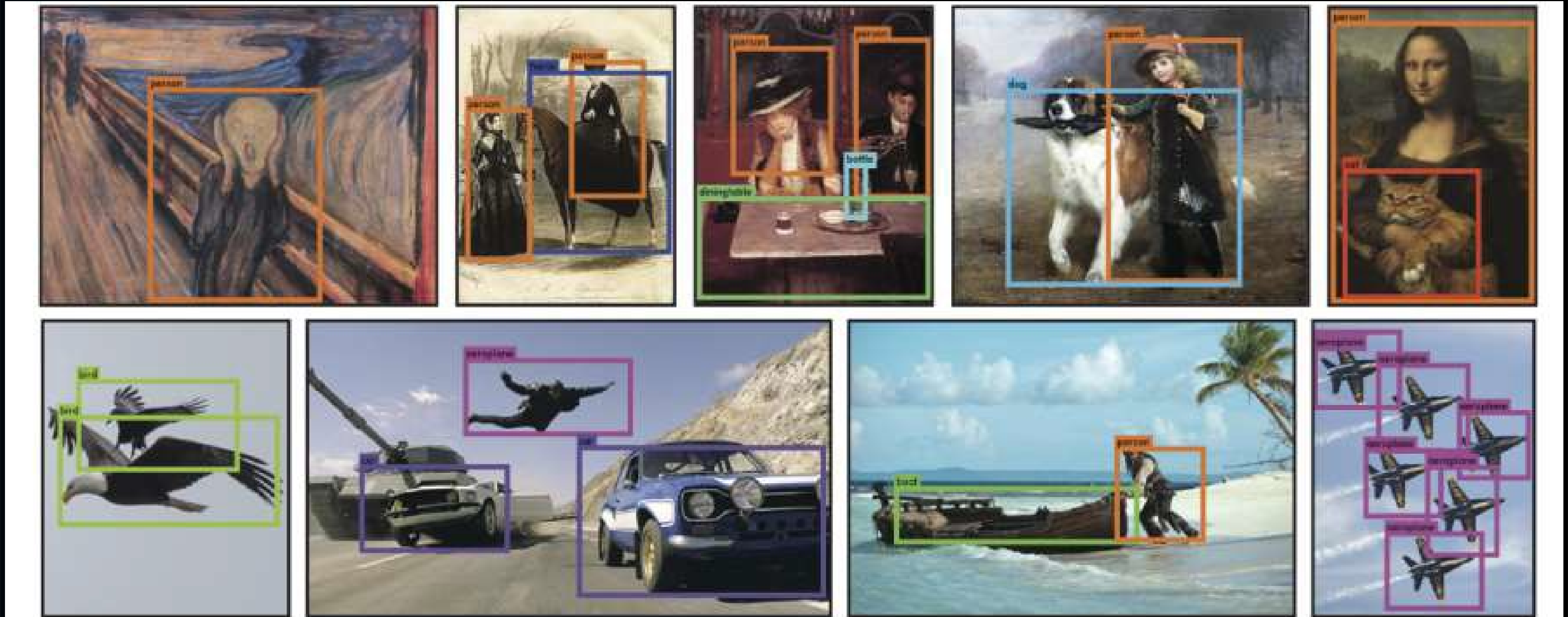
## Cons:

- Each grid cell is responsible for the prediction of two boxes and can have only two classes
- YOLO is pretty bad at predicting groups of small boxes
- YOLO breaks with predicting new, unusual shapes or unusual aspect ratio
- Large boxes matter the same as small boxes in errors calculation
- It's particularly impossible to find all the objects for natural images (according to two maximum two boxes per cell constraint)
- Unstable gradient in early stages of training
- No batch normalization at all
- YOLO trains the classifier with 224x224 images and then moves to 448x448 images for object detection





# Person Detection in Artwork



**Figure 6: Qualitative Results.** YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

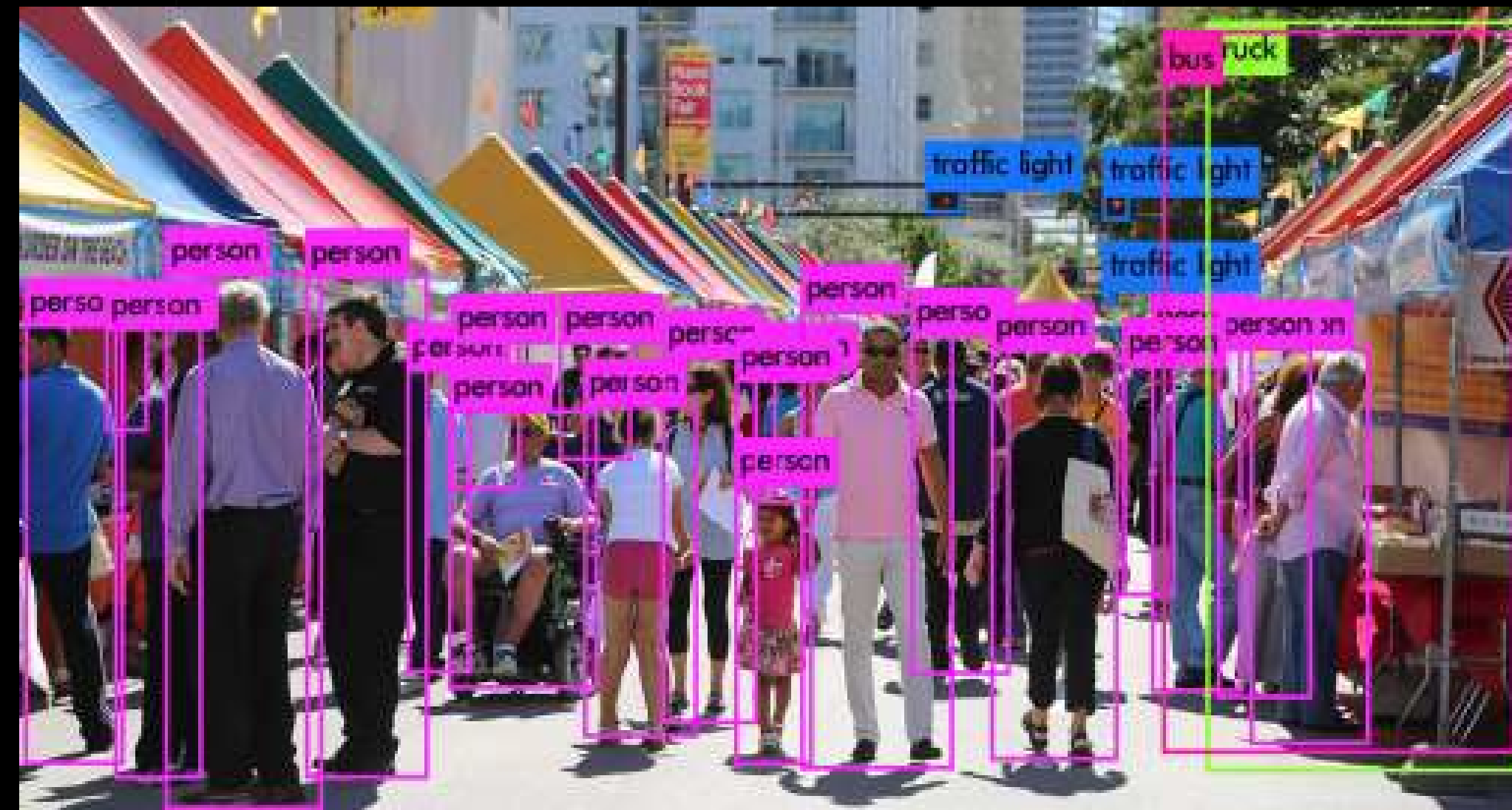
21 October  
2025







# Darknet (YOLO) results on random images



21 October  
2025





Goals of Day 3

# Gesture Recognition

# Introduction to Machine Learning & YOLO

# Sneak Preview Digital Twin

21 October  
2025







# Deep Learning for Computer Vision Tasks with custom data sets

21 October  
2025





# Real World Data is Messy. How to write an algorithm that handles all edge cases?

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



Intra-class variation







# Data-Driven Approach (Classical) vs. Deep Learning Approach in Training Computer Vision Models

21 October  
2025





# Data-Driven Approach

- relies heavily on a robust, annotated dataset
- Traditional machine learning models like haarcascades with manual feature extraction and engineering
- Quality of model is closely tied to the quality of data it's trained on
- Simple, Transparent and reliable

Training set (~80%)

Test set (~20%)

21 October  
2025



# Deep-Learning approach

- A subset of machine learning where algorithms are inspired by the structure and function of the brain's neural networks
- Deep learning models automatically learn features from raw data, eliminating the need for manual feature extraction
- increased capacity for complexity in model structures and the ability to handle large-scale, high-dimensional data is needed

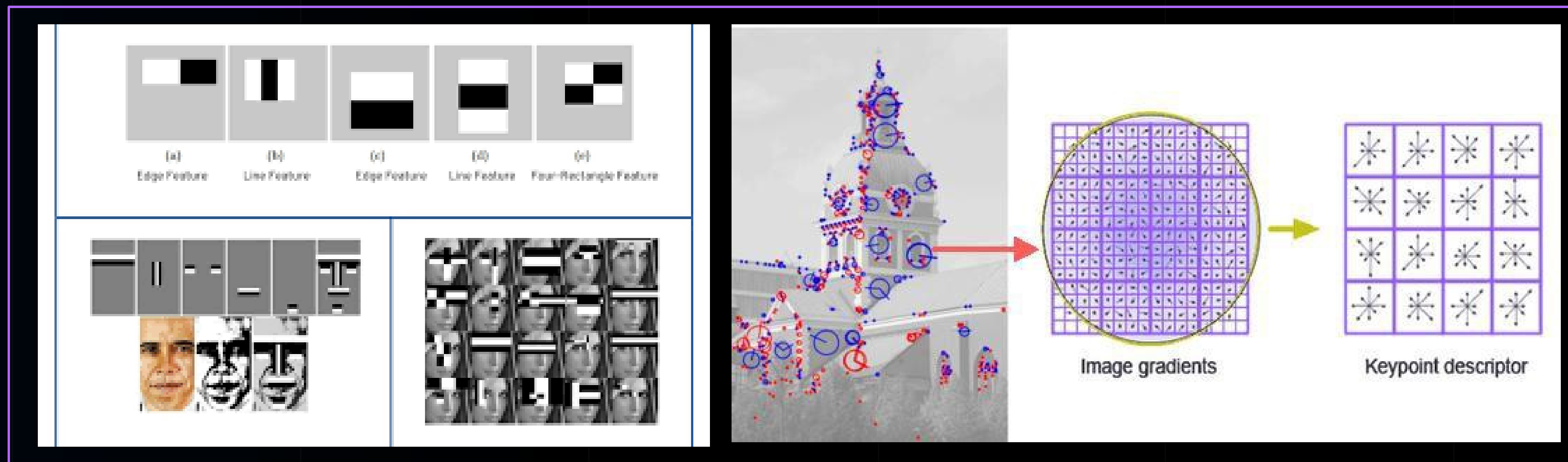
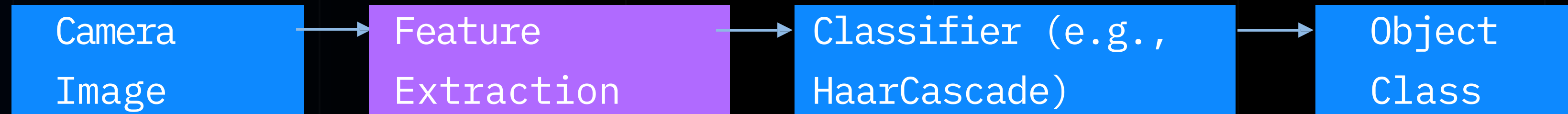
Training set (~80%)

Test set (~20%)





# Computer Vision Pipeline – Before Deep Learning





# Data-Driven Approach – A Day in a Life

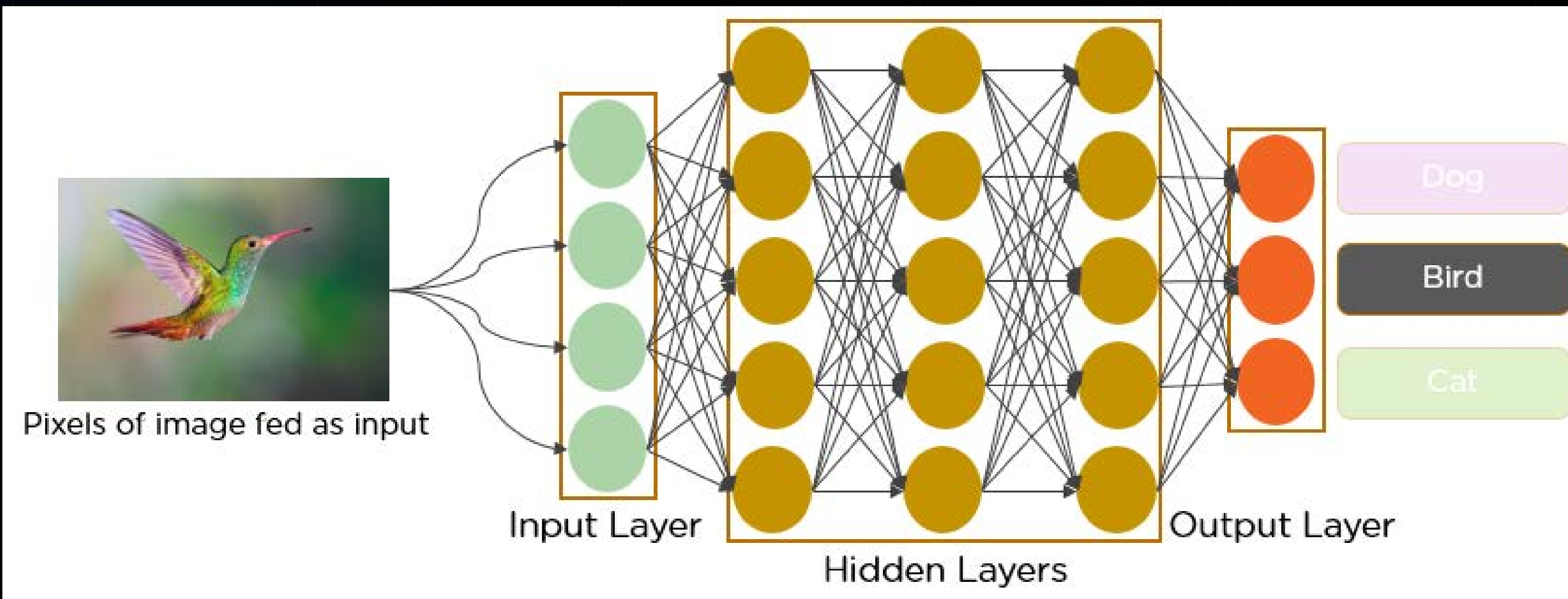
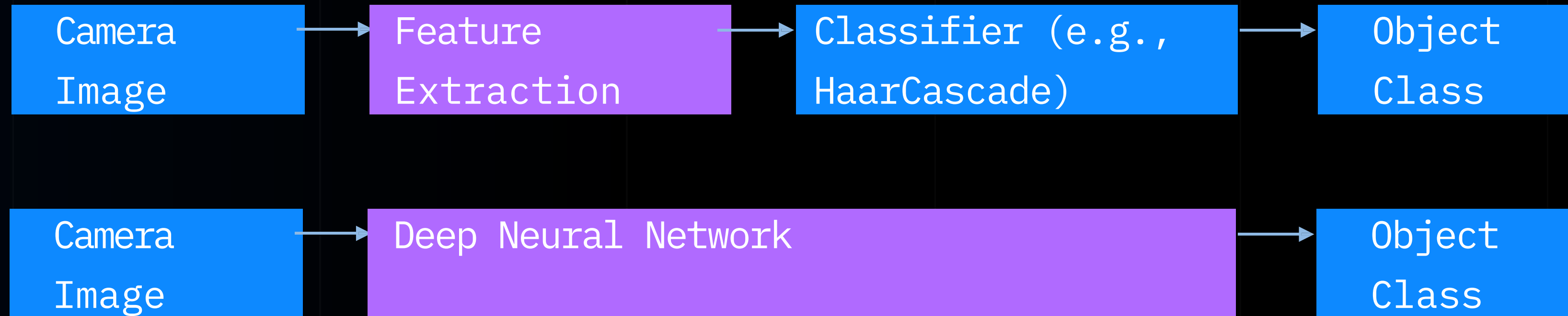
1. Data Collection
2. Data Cleaning & Preprocessing
3. Data Exploration & Analysis
4. Feature Engineering & Selection
5. Modeling
6. Evaluation







# Deep Learning Computer Vision Pipeline



21 October  
2025



# Deep Learning Approach – A Day in a Life

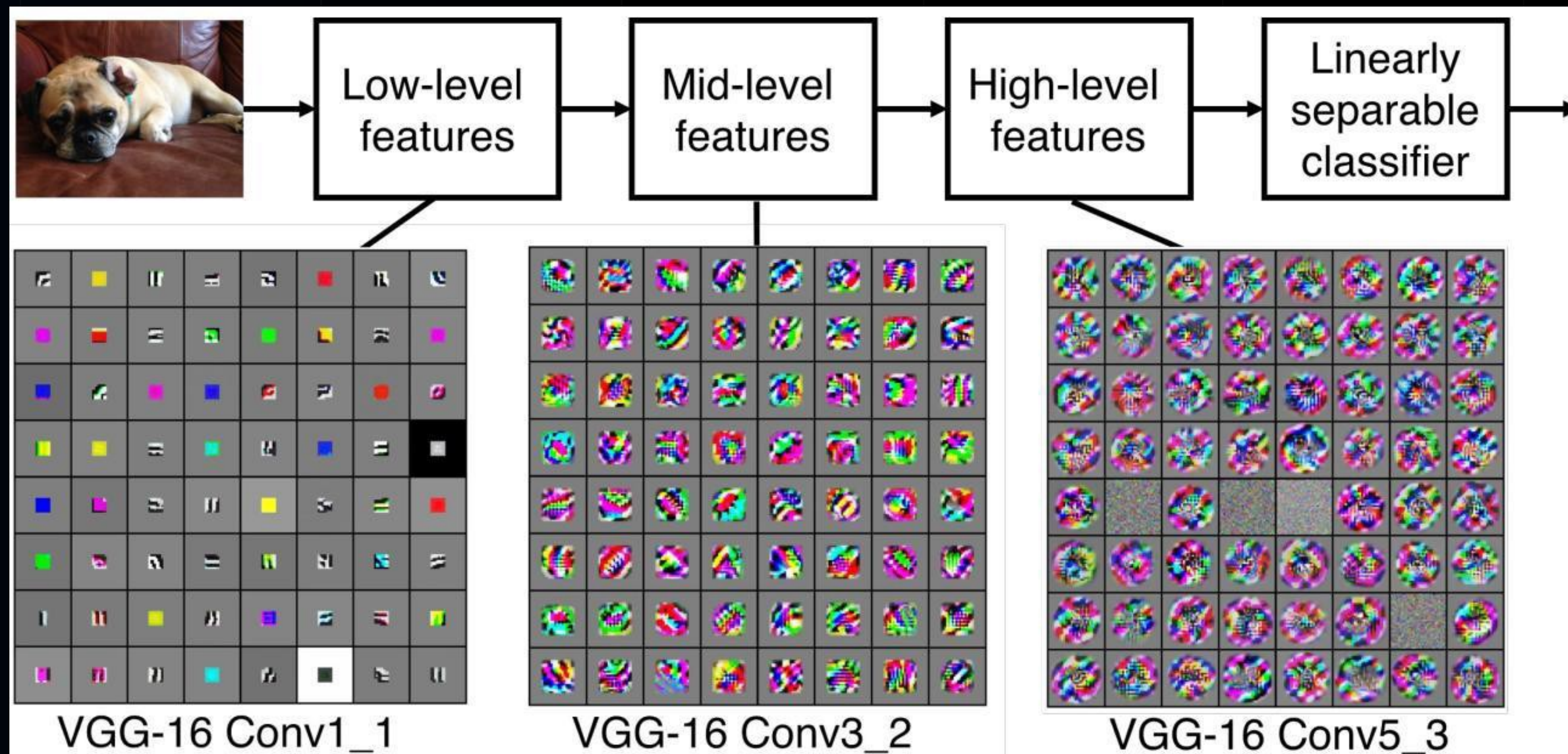
1. Data Collection
2. Data Preprocessing & Augmentation
3. Model Design
4. Training
5. Validate & Tune
6. Deploy Model
7. Display







Hidden Layers are like Filters that react to specific features (edges, faces, hands, people, smiles...) becoming more complex the deeper it is in the neural network



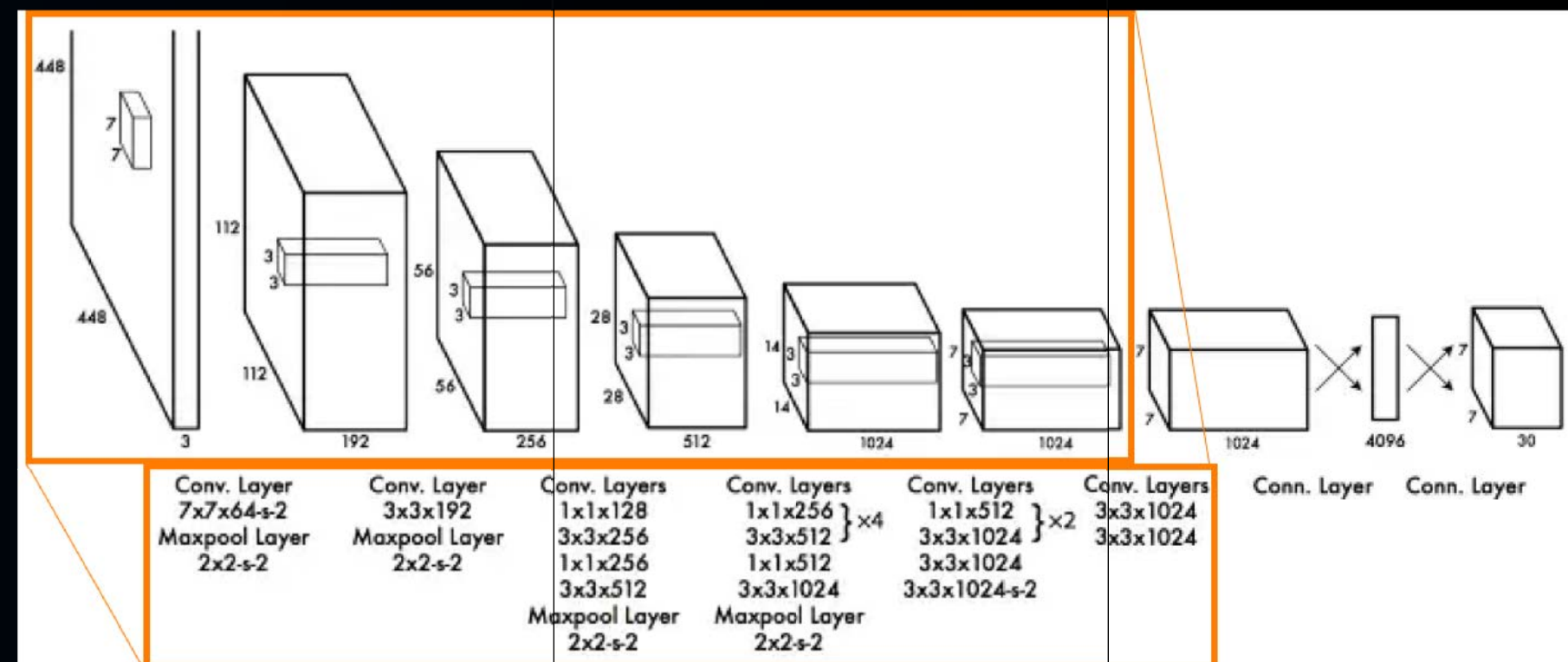
21 October  
2025



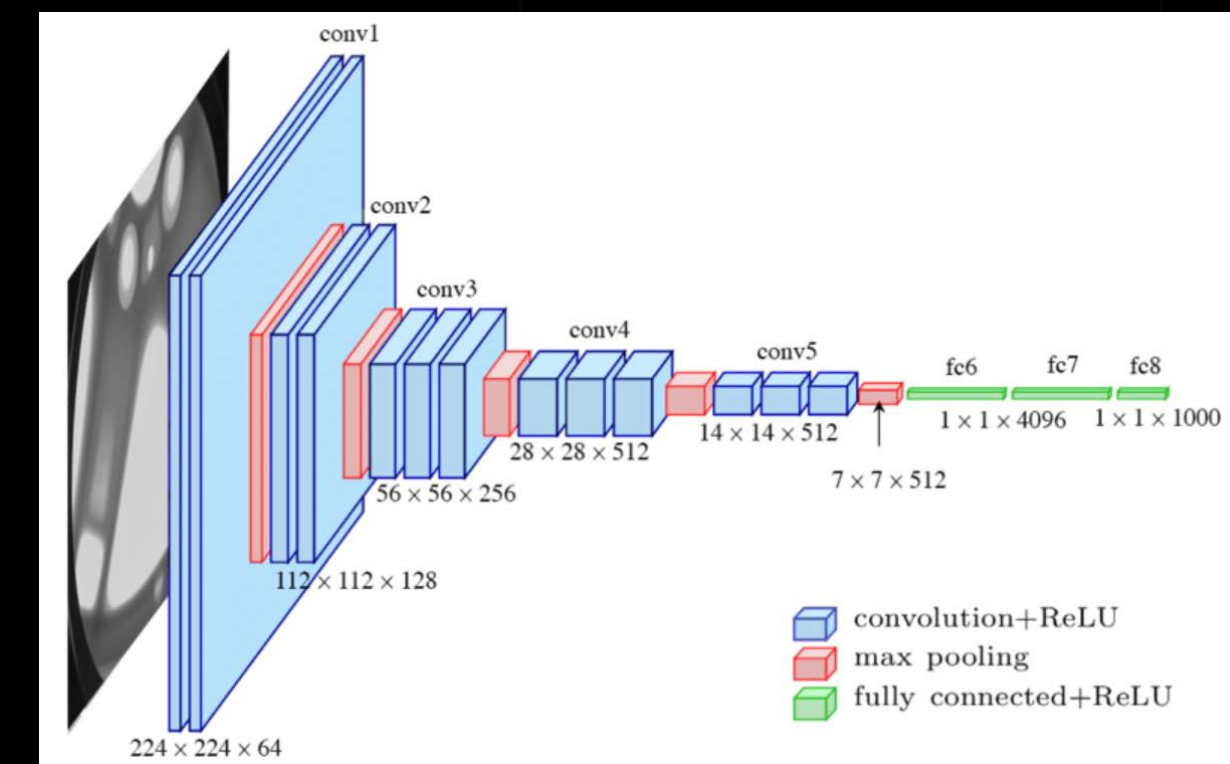


# CNN Architectures

- You will come across a couple of Pyramid Shaped Network Designs of CNN each with its own strengths, improvements & specific use-case areas: <https://vitalflux.com/different-types-of-cnn-architectures-explained-examples/>

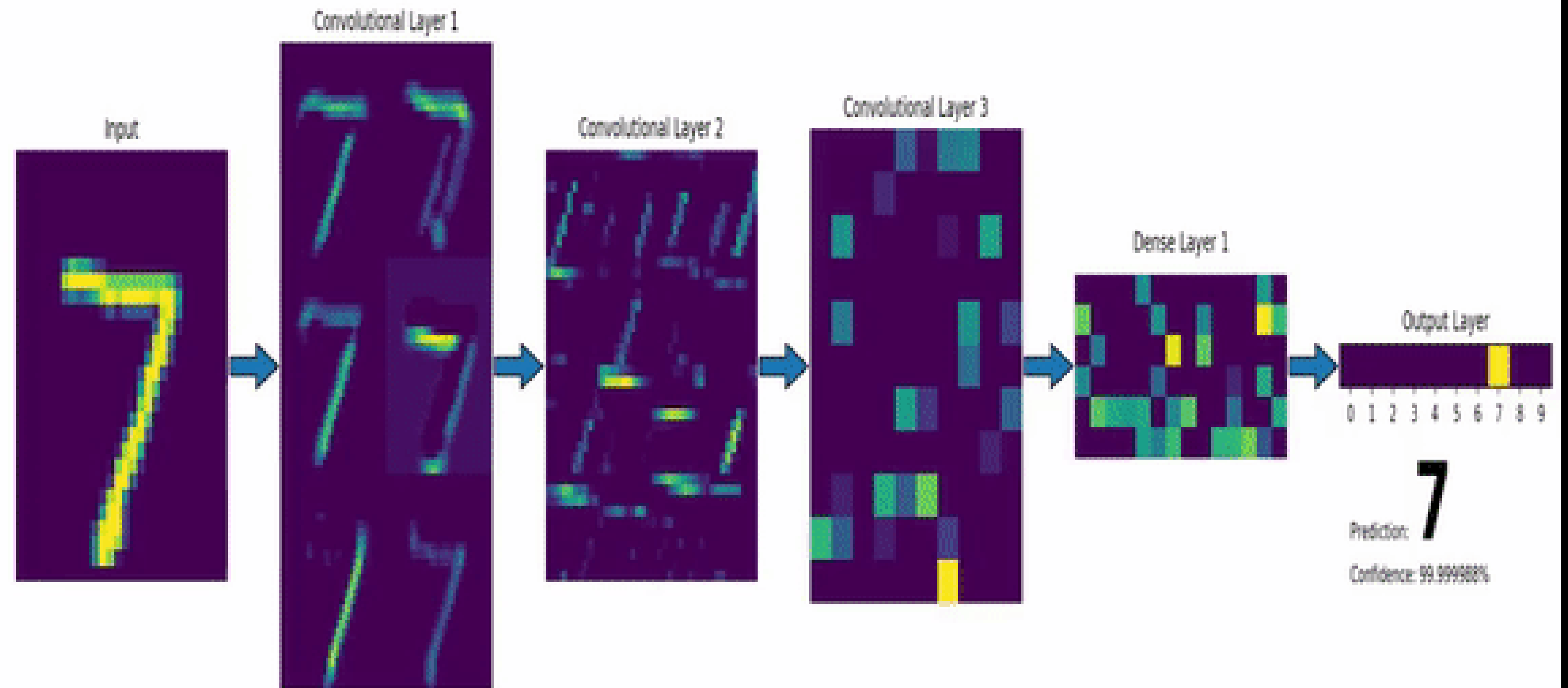


YOLO: 24 convolutional layers that "stretch" an image with size (448x448x3: w,h, RGB channels) into a tensor with the size of (SxSx1024, where S=7).



VGGNet: 16-layer CNN with up to 95 million parameters and trained on over one billion images (1000 classes). It can take large input images of 224 x 224-pixel size for which it has 4096 convolutional features







We're really just scratching the surface

# Layer Types

# Losses

# Training

- Pooling
- Upsampling
- Batching
- Dropout/Regularization

- Loss for Semantic Segmentation
- Loss for Object Detection (Bounding Boxes)
- ...

- Transfer Learning
- ...

21 October  
2025







# How to Start

**Open Datasets:** MS Coco, ImageNet, Cityscapes, KITTI, Apollo, Oxford RoboCar, ...

**Open-source culture:** sharing-is-caring many papers release code on GitHub & Huggingface

**Frameworks:** Tensorflow, Keras, PyTorch, YOLO and GPU for Training/Inference



21 October  
2025







# Data Preparation & Analysis

21 October  
2025





Machine Learning Models  
rely on annotated data, **as of  
today**, with Human in the loop  
(HILO)

21 October  
2025







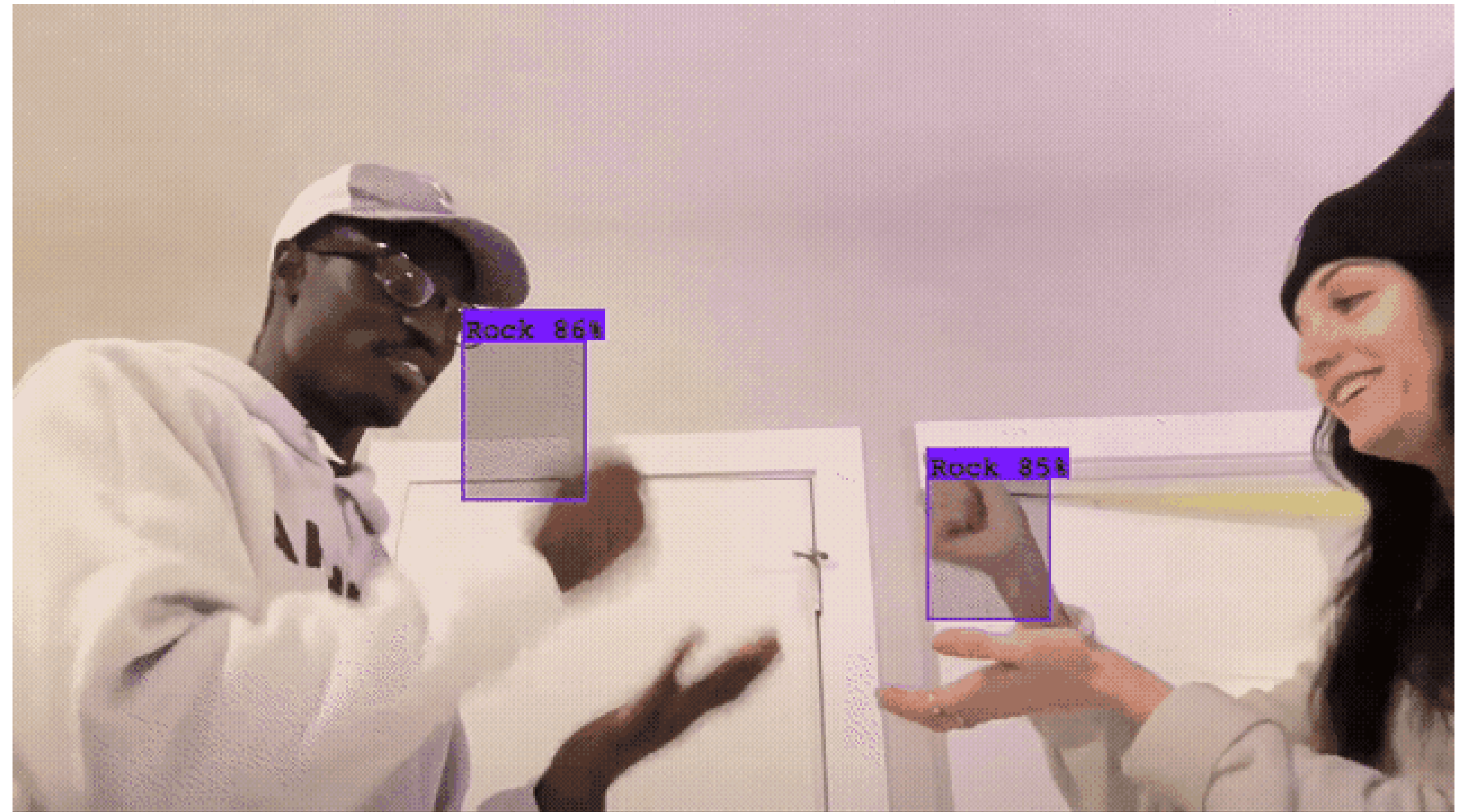
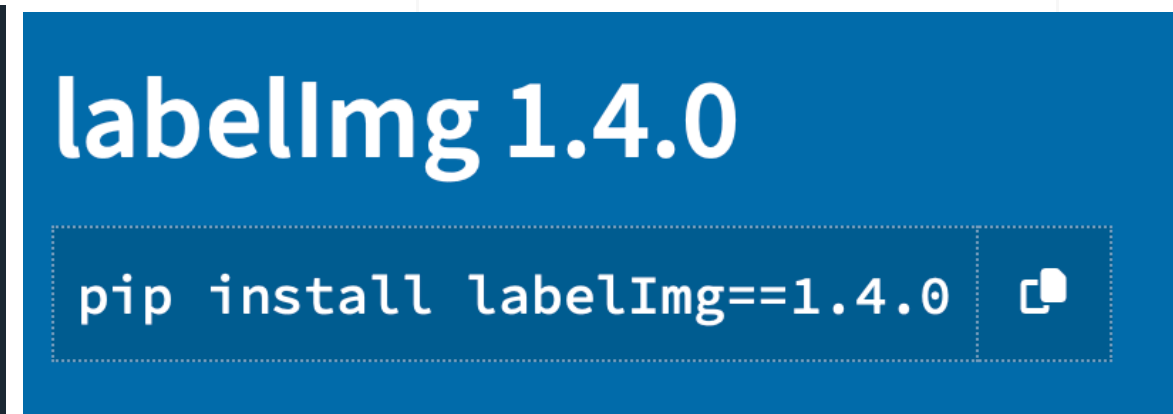
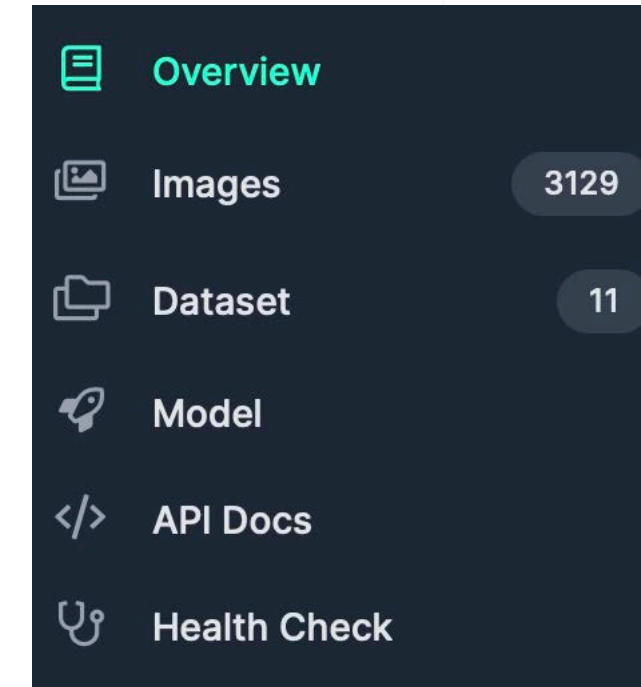
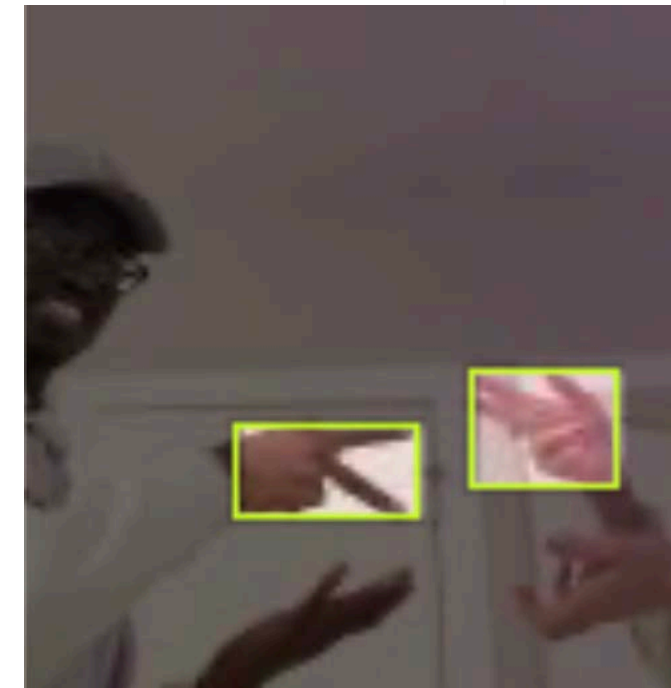
# Annotation Tools

## Tools to label data:

Roboflow  
Labelling  
Microsoft VoTT  
IBM Watson  
AWS  
CVAT

...

21 October  
2025

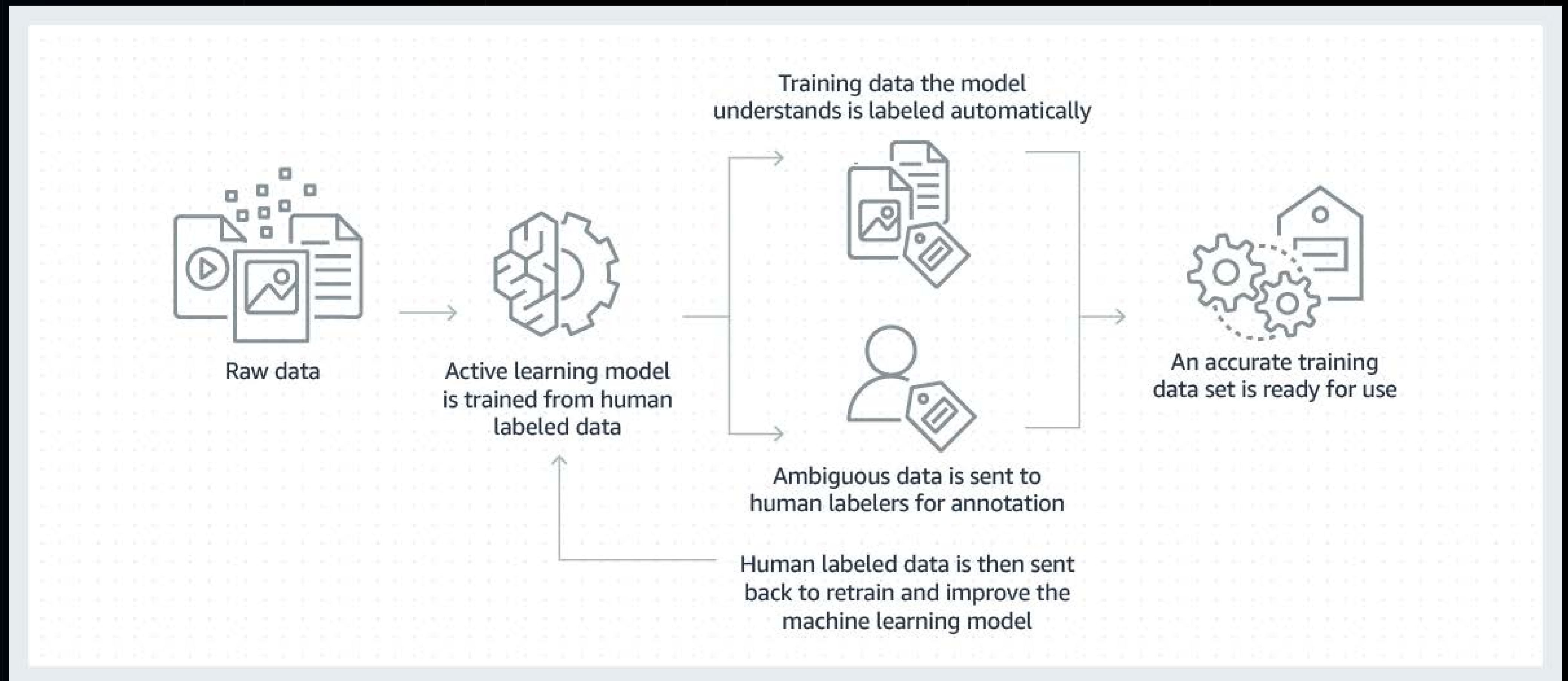


(good practices)

<https://universe.roboflow.com/roboflow-58fyf/rock-paper-scissors-sxsw>



# ML based Data Labelling for higher efficiency





# Key aspects for obtaining good results

Dataset Volume

Variety of the dataset (position, angles, distances)

Dataset quality (context, scene)

Image size (medium resolution)

Data augmentation

Labeling of images (good practices)

21 October  
2025

(good practices)







# Preparing your Dataset

1. Check your data before training
2. Check your data visually
3. Check data statistics
4. Check image labels/ annotations

21 October  
2025



<https://www.youtube.com/watch?v=40GqhTrMcNA>



# Good Labelling Practices

- Look at every single image. Labels can be misleading and unexpected. Check them carefully

Training should have enough variations. Do not oversample from the same video

Frames from the same video should not be part of the training and validation set to avoid bias,

- Check training / validation split, and volume based on the problem

Look for class imbalance

- missing objects, Missclassified label , Inconsistent labeling

21 October  
2025

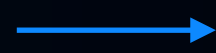


<https://www.youtube.com/watch?v=40GqhTrMcNA>



# Data Augmentation techniques allow models to adapt and work better in a real environment.

Original dataset 9,000 images



Dataset increased 40,000 images

Saturation

Contrast

Rotate

Sharpness

Fuzziness

Tonality

Noise

(good practices)

Cut

Normalize

Contrast

Brightness

21 October  
2025



## Example DataGenerator with Keras

```
from keras.preprocessing.image import ImageDataGenerator

image_gen = ImageDataGenerator(rotation_range=30, # rotate
                                the image 30 degrees
                                width_shift_range=0.1, #
                                Shift the pic width by a max of 10%
                                height_shift_range=0.1, #
                                Shift the pic height by a max of 10%
                                rescale=1/255, # Rescale the
                                image by normalizing it.
                                shear_range=0.2, # Shear
                                means cutting away part of the image (max 20%)
                                zoom_range=0.2, # Zoom in by
                                20% max
                                horizontal_flip=True, # Allo
                                horizontal flipping
                                fill_mode='nearest' # Fill
                                in missing pixels with the nearest filled value
                                )
```





# Annotated Labels

class.txt -> e.g. Cat, Dog, Car, Bus, Truck, Bird  
folder with class images  
folder with image labels are bounding Box with object  
Height, Width, Center (x,y)

21 October  
2025

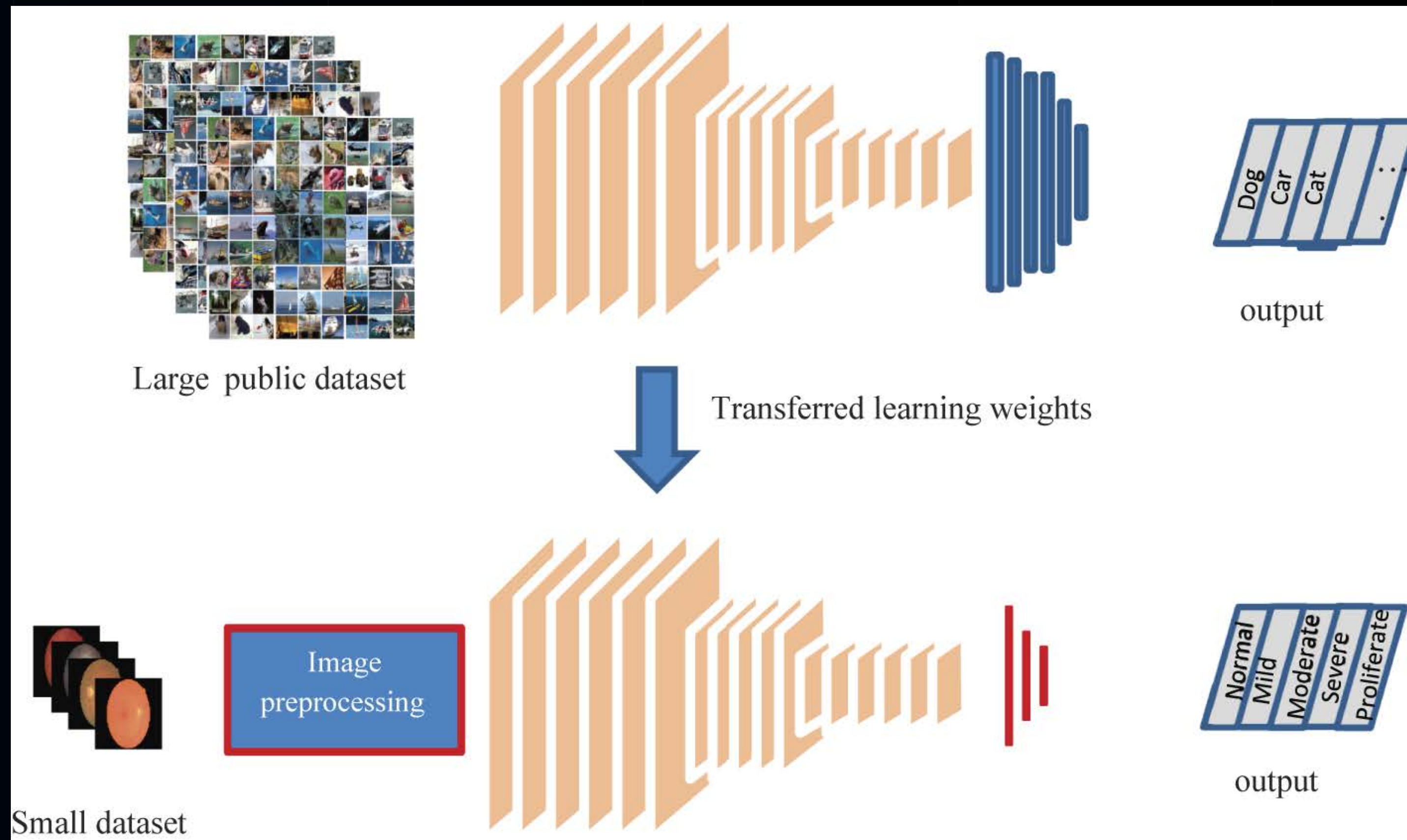


```
└─ yolov8
  └─ train
    ├── images (folder including all training images)
    ├── labels (folder including all training labels)
  └─ test
    ├── images (folder including all testing images)
    ├── labels (folder including all testing labels)
  └─ valid
    ├── images (folder including all testing images)
    └── labels (folder including all testing labels)
```





# Transfer Learning for small datasets



21 October  
2025







# A Keras Transfer Learning Example

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# Load the pre-trained VGG16 model without its top layers
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze the base model
for layer in base_model.layers:
    layer.trainable = False

# Create a new model on top
model = Sequential()
model.add(base_model)
model.add(Flatten()) # Flatten the output layer to 1 dimension
model.add(Dense(1024, activation='relu')) # Add a fully connected layer with 1024 hidden units and ReLU activation
model.add(Dense(1, activation='sigmoid')) # Add a final sigmoid layer for classification

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

21 October  
2025







# Custom Data for YOLO v8 CV tasks

<https://learnopencv.com/train-yolov8-on-custom-dataset/>

```
pip install ultralytics
```

```
pip install clearml
```

21 October  
2025





# Practical Activity 1



21 October  
2025





## Object Detection with YOLO (90 minutes)

Objective: Explore the YOLO (You Only Look Once) object detection system.

### Instructions:

1. **Setup:** Set up your Python environment, import necessary libraries (OpenCV, NumPy), and download the pre-trained YOLO v3 weights and configuration file.
2. **Object Detection:** Load the YOLO model using OpenCV's `dnn.readNetFromDarknet()` function. Use this model to perform object detection on an image or video of your choice, drawing bounding boxes and labels around detected objects.
3. **Exploring Outputs:** Spend time understanding the outputs of the YOLO model, such as the confidence score, class ID, and bounding box coordinates.
4. **Performance Optimization (25 minutes):** Experiment with different parameters, such as the confidence and non-maximum suppression threshold, to optimize the performance of your object detector.

Stretch Goal/Bonus Task: Implement real-time object detection using your webcam. Additionally, explore the differences between YOLO and other object detection models like SSD or Faster R-CNN.





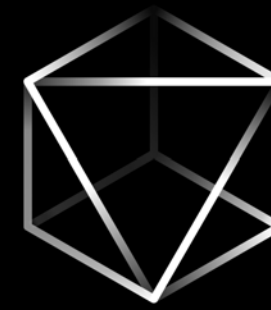


Kick Off Questionnaire

# Thanks & Questions?

21 October  
2025





Aslihan Yilmaz-Hauck | AI & Computer Vision WS25

HSD Düsseldorf | BMI